UNIVERSITY OF
CAMBRIDGE
Department of Computer
Science and Technology

Accelerating Molecular Generation through Representation Alignment

Shreyas Ravishankar

Hughes Hall

June 2026

Submitted in partial fulfillment of the requirements for the
Master of Philosophy in Advanced Computer Science

Total page count: 50

Main chapters (excluding front-matter, references and appendix): 35 pages (pp 7–41)

Main chapters word count: 10380

Methodology used to generate that word count:

> The count is produced with `texcount` over the LaTeX source, following `\input` files. It is scoped to the core chapters by `%TC:ignore` directives that exclude the front matter, bibliography, and appendix. The reported figure sums body text, section headings, and figure/table captions (i.e. narrative text); tabular data listings are not counted, in keeping with the word-count rules. Reproduce with:
>
> ```
> $ make wordcount
> perl tools/texcount.pl -inc -sum -total -1 report-draft.tex
> 10380
> ```

Declaration

I, Shreyas Ravishankar of Hughes Hall, being a candidate for the Master of Philosophy in Advanced Computer Science, hereby declare that this report and the work described in it are my own work, unaided except as may be specified below, and that the report does not contain material that has already been used to any substantial extent for a comparable purpose. In preparation of this report, I adhered to the Department of Computer Science and Technology AI Policy. [The project required the approved use of {insert name of technology} and such use is acknowledged in the text at the relevant sections.] [I am content for my report to be made available to the students and staff of the University.]

Signed:

Date:

Abstract

Write a summary of the whole thing. Make sure it fits on one page.

Contents

Chapter 1

Introduction

The evolution of generative machine learning in scientific domains is defined by the shifting tensions between data scale, data quality, and structural priors. Our report examines a recent addition to this story, *representation alignment*, in two regimes for de novo molecular generation — small molecules and protein backbones — and characterises its behaviour. In this introduction, we trace the narrative arc of research trends in molecular generation, and highlight the motivation for our work: the emerging need for data-efficient modelling in a world of prior-light models.

1.1 A history of trade-offs

The dominant ideas in generative modelling are *denoising diffusion* and its offshoot *flow matching* [1, 2], which found breakthrough success in the realm of image generation. This paradigm has since been adapted for de novo design challenges across scientific fields, such as generating protein backbones [18, 19, 14], inorganic materials [21], and drug-like small molecules [15]. Training these models is typically expensive in both data and compute, with state-of-the-art image generators routinely training on billions of examples [33]. However, the corpus of available high-quality molecular data sits orders of magnitude below this. Experimental determination is slow and expensive, which means datasets are correspondingly small. For example, protein structure generators have largely been trained on at most half a million structures [14], which may explain why they have not yet matched the maturity of their vision-domain counterparts.

Historically, researchers compensated for the small scale of high-quality molecular data by encoding structural priors directly into model architectures [39]. SE(3)-equivariant architectures, for example, enforce the rotational and translational symmetries expected of molecules by construction [41]. But hard structural priors come with a computational cost, and the operations required to encode geometry are a bottleneck to scaling models.

A recent wave of research has moved entirely the other way: drop hard priors, use bare-bones non-equivariant transformer stacks, and rely on data scale and augmentation to teach the model what equivariance buys. This shift fits a broader trend in machine

learning on leveraging representation instead of model architecture to inject soft inductive biases for structured data. Vision Transformers [42] showed that the spatial-locality and translation-equivariance priors of convolutional neural nets (CNNs) can be learned rather than imposed, given sufficient scale. Graph Transformers like TokenGT [43] have made the same case for graphs. AlphaFold 3 [45] stripped many of the SE(3)-equivariant biases of AlphaFold 2 [44], replacing the structure module with a diffusion model operating on raw coordinates. Brehmer et al. [46] make the empirical scaling case directly: more data through a simpler model beats less data through a richer one. In our work, we examine Tabasco [15] and Proteina [14], which are flow-based generators built on largely vanilla transformers for small molecules and protein backbones respectively.

Given the difficulty of procuring data in scientific domains, data scale must necessarily come from non-experimental sources, which may be of lower quality. For example, recent efforts in protein generation have leaned on synthetic structures from folding-model predictions [14, 35] to expand training datasets into the tens of millions. However, even these enhanced corpora are arguably insufficient. Metagenomic databases catalogue over two billion natural protein sequences [37], and the theoretical sequence space at a median protein length [38] is hundreds of orders of magnitude greater still. Even at the frontier of the field's data scale, we sample but a sliver of the distribution we wish to model.

In summary, the field has traded hard structural priors for architectural simplicity, and curated data for synthetic scale. However, this trade-off dilutes the structural signal available to a model. Furthermore, even our best attempts at creating large datasets may not be large enough. These twin challenges amplify the need for techniques that make training still more efficient without sacrificing quality.

1.2 Representation alignment

But why is training diffusion-transformer architectures expensive in the first place? One answer comes from Yu et al. [5], who argue that these models suffer from a *representation bottleneck*. During training, a model is implicitly asked to learn two distinct tasks simultaneously: (1) representation: extracting semantically meaningful encodings from complex data; and (2) generation: constructing data from these encodings. The standard denoising objective alone may not provide sufficient signal for representation learning, leading to slow convergence and the large training budgets these models require in practice. In the context of molecular generation, this representation burden is compounded by the twin challenges outlined above. Models are now forced to encode complex structural geometry from datasets that only sparsely sample the true underlying distribution, all while operating without the aid of architectural priors.

Yu et al. propose Representation Alignment (REPA) as a remedy. The technique adds a regularisation term that aligns a generator's intermediate hidden states with the embeddings of a frozen pretrained encoder. On image diffusion, they report roughly a 17.5×

training speed-up against a strong baseline to achieve similar generation quality. REPA has since been extended beyond images to video [9], audio [10], and recently to molecular generation [7, 8], where it has been demonstrated but not systematically characterised.

1.3 Contributions

In this light, the core question we investigate in this report is: *can prior-light molecular generation models benefit from representation alignment with pre-trained models, in training efficiency and generation quality?* We aim to characterise when and why this works, including failure modes and limitations.

We document two lines of enquiry.

(i) *Does REPA work in the molecular domain?*: We conduct empirical studies on training flow-based generators with REPA in two regimes: Tabasco [15] for small molecules, and Proteina [14] for protein backbones. We find that REPA does not measurably improve Tabasco, but it improves training efficiency and advances the generation quality envelope for Proteina.

(ii) *When does REPA work?*: Following Singh et al. [6] in the realm of image generation, we develop a profiling framework to characterise the suitability of an encoder as a representation target. We measure three properties of an encoder: (1) an upper bound on the structural information it contains (linear-probe recoverability against domain structural labels); (2) a lower bound on what a projector can transfer from that information (projector-based recoverability); (3) the geometric conditioning of its embeddings for the alignment objective (effective rank).

We do not propose a new architecture or alignment loss, and our two studies are not a controlled experiment over the conditions that govern REPA's effectiveness. Instead, they are best read as paired case studies in regimes where those conditions differ.

The remainder of this report is organised as follows. Chapter 2 sets out the background and context needed to read the rest: flow matching, REPA, molecular generation. Chapter 3 discusses what it means for a generative model to be "good" in our domain, and the tensions that question hides. Chapter 4 develops the encoder-profiling framework and uses it to predict where REPA may and may not help. Chapters 5 and 6 present the two paired studies, small molecules first and protein backbones second. Finally, we conclude in Chapter 7 with a discussion of whether what we learn generalises beyond molecular generation.

Chapter 2

Background and Related Work

In this chapter, we introduce the machinery the rest of this thesis depends on. We outline flow matching (§2.1), the generative paradigm underlying the models we study, and representation alignment (§2.2), the regulariser that is our object of study. We then turn to the molecular setting. We situate generative models within the broader de novo molecular design pipeline (§2.3), before presenting the two molecular domains we work in, small molecules and protein backbones, and the flow-matching models we extend, Tabasco and Proteina (§2.4). The treatment is cursory by design. We intend only to guide the reader through this report, and we provide citations for omitted details.

2.1 Flow matching

At first glance, the term *generative modelling* implies the problem of *creating* new artefacts from scratch. However, it is perhaps more accurately described as the problem of *sampling* new artefacts from a complex data distribution that can only be accessed through a finite training set. A common strategy here is to fix a simple distribution we can sample from cheaply — typically standard Gaussian noise $p_0 = \mathcal{N}(0, I)$ — and learn a mapping from it to the desired data distribution p_1.

Flow matching (FM) [2] realises this mapping as a continuous-time transport. The model designer defines a *probability path* between a noise sample and a data sample — a continuous interpolation between them — and the model learns to carry the noise sample along this path. Concretely, it parameterises a time-dependent *velocity field* $v_\theta(x, t)$ that gives the instantaneous direction of motion at each location x and time $t \in [0, 1]$. At inference, integrating the learned ordinary differential equation (ODE) $\dot{x}_t = v_\theta(x_t, t)$ from $t = 0$ to $t = 1$ pushes a fresh noise sample forward into a fresh data sample.

A common choice of probability path is the linear interpolant, $x_t = (1 - t)\, x_0 + t\, x_1$ for $x_0 \sim p_0$ and $x_1 \sim p_1$. The model's target is the *marginal* velocity field at (x, t): the average direction of motion over all noise-data pairs whose paths pass through that point. However, supervising this marginal directly is intractable, since the contributing pairs are not known in advance. Flow matching sidesteps this difficulty with a key observation [2]:

although the marginal field may be intractable, the *conditional* velocity along the path of any specific noise-data pair is relatively trivial. Indeed, for the linear interpolant, it is simply $\dot{x}_t = x_1 - x_0$. Regressing the model on these conditional velocities, averaged over many sampled pairs, recovers the marginal field in expectation, and gives the training loss as a mean-squared regression:

$$\mathcal{L}_{\mathrm{FM}} \;=\; \mathbb{E}_{t,\,x_0,\,x_1} \left\| v_\theta(x_t, t) - (x_1 - x_0) \right\|^2 .$$

In practice, every training step samples a time $t \in [0, 1]$ and a noise-data pair (x_0, x_1), computes the interpolant x_t, and takes a gradient step on this squared error.

Flow matching with this linear interpolant generalises score-based diffusion: the variance-preserving Gaussian-path objective of denoising diffusion models [1] is a special case of the FM family [2, 4].

Generating samples from a trained flow-matching model amounts to integrating the learned velocity field. This is often performed with a standard ODE solver, with Euler steps usually sufficing for stable trajectories. The field also admits stochastic sampling variants — such as those drawing on the connection to score-based stochastic differential equations (SDEs) — which inject noise during the integration steps to explore the probability space and improve sample diversity. The amount of injected noise is itself a tunable knob: more noise broadens diversity at some cost to per-sample quality, with the deterministic ODE recovered as its zero-noise limit. The models we work with in this report, Tabasco and Proteina, support both deterministic and stochastic samplers.

Conditional generation augments v_θ with context tokens — such as a class label, a sequence length, or a structural motif — with conditioning supplied at both training and sampling time. In this report, we focus on unconditional generation only.

For a more complete treatment of flow matching, we refer the reader to Lipman et al. [2] for the original presentation, and to the recent Lipman et al. guide [4].

2.2 Representation alignment

Chapter 1 introduced the argument made by Yu et al. [5] about the *representation bottleneck* in diffusion-transformer models. Their analysis applies to flow-matching models as well, given the equivalence between the two objectives discussed above (§2.1). The flow-matching objective implicitly asks a model to learn two things at once: a useful representation of the data, and a velocity field that interprets this representation. This twin burden, they argue, leads to slow convergence and large training budgets.

Their proposed solution, Representation Alignment (REPA), addresses the first half of this burden by adding a regularisation term to the training loss. Figure 2.1 shows the construction. The regularisation pulls z_ℓ, the generator's intermediate hidden state at a chosen *alignment layer* ℓ, towards the embeddings produced by a frozen pretrained

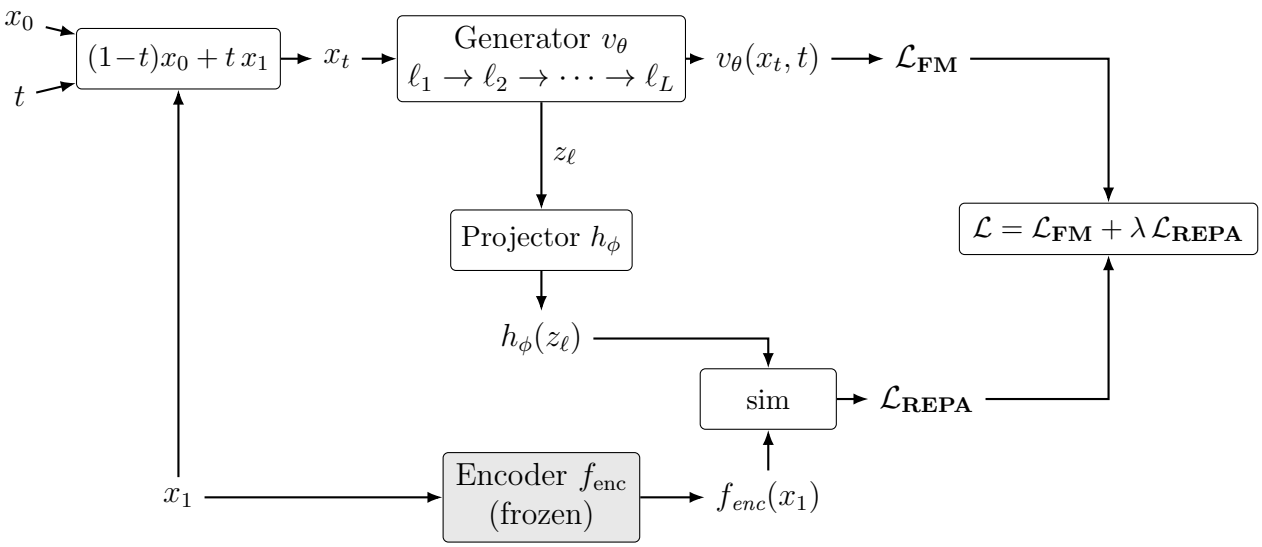

Figure 2.1: The REPA construction. A noised sample $x_t = (1-t)x_0 + t\,x_1$ is constructed at each training step from a noise sample x_0, a clean data sample x_1, and a sampled timestep $t \in [0, 1]$. The generator v_θ processes x_t through its sequence of layers $\ell_1, \ldots, \ell_L$ to produce the velocity field that feeds the flow-matching loss $\mathcal{L}_{\text{FM}}$. Separately, the clean sample x_1 is also passed through a frozen pretrained encoder f_{enc} (shaded). The generator's hidden state z_ℓ at the chosen alignment layer is mapped into the encoder's feature space by a trainable projector h_ϕ; the REPA loss $\mathcal{L}_{\text{REPA}}$ is the negated similarity between $h_\phi(z_\ell)$ and $f_{\text{enc}}(x_1)$, averaged across tokens. The total training loss $\mathcal{L}$ combines them with weight λ.

encoder f_{enc} run on the clean data sample x_1. The hidden state z_ℓ is a sequence of N token vectors. In the context of images, each token represents an image patch; in the molecular setting, each token represents an atom or residue that is part of the larger molecule (§2.3). A small trainable *projector head* h_ϕ — a three-layer multi-layer perceptron (MLP) in the original paper — maps z_ℓ into the encoder's feature space, so that $h_\phi(z_\ell)$ and $f_{\text{enc}}(x_1)$ can be compared. This alignment is asymmetric. The encoder provides a fixed target throughout training, while the generator parameters θ and the projector ϕ are updated.

The alignment loss is the negated similarity, averaged token-wise, between projected and target embeddings:

$$\mathcal{L}_{\text{REPA}} \;=\; -\,\mathbb{E}_{t,\,x_0,\,x_1}\left[\frac{1}{N}\sum_{n=1}^{N}\text{sim}\big(h_\phi(z_\ell)_n,\; f_{\text{enc}}(x_1)_n\big)\right],$$

where n indexes the N tokens within a sample. In the image setting, N is constant across the dataset, so this token-wise average is equivalent to averaging similarities once per sample and then across the batch. Indeed, the original paper draws no distinction between the two. For molecular data, where N varies per sample, the two diverge. We return to this distinction in the design knobs below.

The final training objective is the flow-matching loss from §2.1 plus this term, scaled by a weight λ:

$$\mathcal{L} \;=\; \mathcal{L}_{\text{FM}} \;+\; \lambda\,\mathcal{L}_{\text{REPA}}.$$

This construction exposes several design choices, some of whose effects we revisit in the ablations of Chapter 6:

- The choice of encoder f_{enc}. Different encoders may capture distinct aspects of the data distribution.
- The alignment layer ℓ at which we attach the projector. Yu et al. experimentally determined that mid-network layers work best for image diffusion, but the optimal choice is not obvious a priori.
- The alignment-loss weight λ, which balances the alignment signal against the generative objective. Once again, Yu et al. experimentally determined an optimal value of 0.5 for image diffusion, but this may not hold universally.
- The choice of similarity metric sim. Yu et al. compared cosine similarity and mean-squared error for image diffusion, and found the former more effective. We adopt cosine similarity in this report as well.
- The averaging mode. Averaging tokens within each sample first weights every molecule equally, while pooling tokens across the batch gives larger molecules proportionally more REPA signal.

We adopt the core REPA construction as in Yu et al. for this report. However, a small family of methodological variants on REPA has since emerged. Dispersive regularisers [12] replace the external encoder with an internal contrastive term, while flexible-target formulations [7] broaden which encoder features the loss attaches to. Separately, REPA-style alignment has been applied beyond the image setting, to video [9], audio [10], materials [11], and molecules [8]. We defer exploration of these variants to future work.

2.2.1 What makes for a good REPA encoder?

There are many possible choices for the encoder f_{enc} in any REPA construction, and this selection is a key design decision. But what characteristics make for a good alignment target? In their original report, Yu et al. [5] observed that ImageNet linear-probe accuracy correlates positively with generation performance, suggesting that global semantic representation is the driving factor. However, more recent work by Singh et al. [6] reports the opposite after profiling a wider range of encoders. Linear-probe accuracy does not in fact predict REPA gain, they argue, while a measure of the spatial structure of patch-token representations does. This suggests that generation performance for REPA may be driven by *local* feature geometry, rather than *global* semantic strength. Encoder characterisation for REPA remains an open research question, particularly in the molecular generation setting, and we pick up the thread in Chapter 4.

2.3 A primer on molecular generation

Having outlined the modelling paradigms driving our report, we now turn to the molecular setting in which we apply them. To situate our work, we begin with a brief primer on

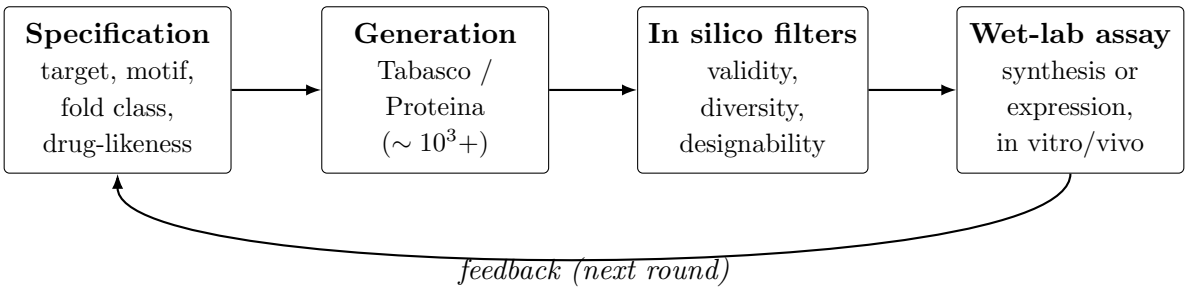

Figure 2.2: The de novo molecular design cycle. An upstream specification drives a generative model to produce a candidate pool, which is funnelled through in silico filters and then synthesised (or expressed) and assayed in the wet lab. Experimental results feed back into the next round of generation. Typical generation-step pool sizes are in the thousands to tens of thousands of candidates per round [18].

the role generative models play within the broader de novo molecular design pipeline. Generative models are rarely intended to produce ready-to-use artefacts. Instead, they act as high-throughput in silico proposal engines in an iterative process of refinement and selection. This process, often called a *design campaign* [54], is summarised in Figure 2.2.

An upstream specification, such as a drug-likeness profile or a fold class, conditions a generative model to produce a pool of candidate samples. These pools typically comprise thousands to tens of thousands of candidates per round [18], which are then funnelled through in silico filters. For small molecules, these typically include checks for structural validity and physical plausibility [55], drug-likeness [57], and target docking; for protein backbones, designability and structural diversity are the dominant considerations [14]. The select few survivors are produced in a laboratory, through chemical synthesis (small molecules) or expression in cell lines (proteins), and evaluated using in vitro or in vivo assays. Experimental results then feed back into the next round of generation.

This pipeline framing is relevant to our discussion of evaluation metrics in Chapter 3. Several metrics we adopt, such as structural validity and designability. are not measures of intrinsic molecular quality, but proxies for specific in silico filter steps.

2.4 Tabasco and Proteina

The two models we study — Tabasco [15] for small molecules and Proteina [14] for protein backbones — are both non-equivariant transformer flow-matching generators. They are explicitly designed to scale with data and model size, rather than relying on structural priors imposed by model architecture. The rotational and translational symmetries expected of 3D molecules are intended to be learned from data and augmentation rather than imposed by construction.

Tabasco is motivated by the success of similar architectures in protein-structure generation, like Proteina, setting it apart from equivariant message-passing small-molecule generators such as EQGAT-diff [49], MiDi [50], and SemlaFlow [51]. In turn, Proteina

is motivated by the pair-bias attention mechanism introduced in AlphaFold 3 [45], which distinguishes it from SE(3)-equivariant backbone generators such as RFdiffusion [18], FrameFlow [47], and Genie [48]. Despite their architectural minimalism, both achieve performance comparable or superior to equivariant baselines on structural-quality evaluations.

2.4.1 Small molecules and Tabasco

In medicinal chemistry, a *small molecule* is a drug-like compound at the sub-900-Dalton scale [52]. In silico, its structure is typically represented using two coupled features: a discrete *molecular graph* of typed atoms connected by typed bonds, and a continuous *conformer* placing those atoms in 3D Cartesian space. A flow-matching parameterisation must therefore handle both jointly.

Tabasco [15] represents each molecule as a sequence of per-atom tokens, each carrying an atom-type embedding and a 3D coordinate, but with no explicit bond decoder. This treats the molecule as a point cloud of typed atoms rather than a graph, sidestepping edge modelling at the cost of having to recover bond structure post hoc from interatomic distances. Tabasco is trained on GEOM-drugs [36], a public dataset of drug-like conformers.

This point-cloud-of-atoms parameterisation is one of several options in the literature, and each makes a different trade-off. *SMILES* and *SELFIES* are string encodings that work well with autoregressive models but discard the 3D conformer entirely; we encounter them later as the parameterisation employed by some candidate encoders we evaluate in Chapter 4. *3D voxel grids* preserve geometry, but lose the discrete graph and inflate the token count by an order of magnitude or more. *Equivariant point clouds*, used by generators such as EQGAT-diff [49], MiDi [50], and SemlaFlow [51], keep both graph and geometry, but pay the computational cost associated with SE(3)-equivariant layers. We refer the reader to Duval et al. [40] and Wang et al. [22] for broader surveys in this space.

2.4.2 Protein backbones and Proteina

A *protein* is a chain of amino-acid residues, ranging from around 50 to several thousands in length [53]; the largest known, titin, has approximately 34,000 residues. (Shorter chains are typically classified as *peptides* instead.) Each amino-acid residue contributes four *backbone* atoms (N, C_α, C, O) whose interactions determine the chain's *fold* geometry, and a *side chain* that determines its amino-acid identity.

A protein's *secondary structure* is its local pattern of α-helices and β-strands; its *tertiary structure* is the global three-dimensional fold those elements form when they pack together. A CATH code [60] (Class, Architecture, Topology, Homology) classifies this fold hierarchically. *Class* captures broad secondary-structure content (mostly-α, mostly-β, or mixed), *Architecture* the gross spatial arrangement of those elements, and *Topology* their specific connectivity.

For the structure-generation problem we study, it is customary to only model the backbone, with sequence identity recovered downstream using a separate *inverse-folding* model such as ProteinMPNN [24]. A further compression is the *CA-trace*, which uses only the set of C_α coordinates for every residue to represent the entire protein. This parameterisation retains enough information to recover fold topology and reduces computational cost relative to the more complete representation, which is typically referred to as *all-atom* modelling. Proteina [14] adopts this C_α parameterisation, representing each backbone as a sequence of per-residue tokens carrying continuous C_α Cartesian coordinates. The models reported in the original presentation are largely trained on high-confidence synthetic structures from the AlphaFold Database (AFDB) [35].

Once again, the CA-trace parameterisation is one of several techniques used in the literature to parameterise proteins in silico. *Sequence-only* models, such as the ESM lineage [23], operate on amino-acid identity strings. These models capture rich functional priors, but encode far less information on 3D structure. *Full-atom* representations close the gap to physical realism, but add substantial compute cost. *Equivariant* parameterisations, such as torsion angles and oriented residue frames, encode SE(3) symmetries by construction, and form the standard basis for the equivariant generators noted above. However, these architectures have historically bottlenecked scaling. We refer the reader to Notin et al. [58] for a broader survey of machine-learning approaches to protein design.

2.5 Open questions

This survey leaves two questions open that the rest of this report takes on:

(i) *Does REPA work in the molecular domain?* Existing applications of REPA in molecular generation [7, 8] have not been ablated over encoder choice, and protein backbone generation in particular remains unexplored.

(ii) *When does REPA work?* The encoder-characterisation work surveyed in §2.2.1 has begun to ask which properties of an encoder predict REPA gain, but only within the image setting. We extend this question to the molecular domain.

We take both questions up empirically in Chapters 4–6.

Chapter 3

Evaluating molecular generation

Chapter 4

Encoder targets and profiling

Chapter 5

REPA on small molecules

Previously (Chapter 4), we profiled various encoders in the molecular generation domain for their suitability as a representation alignment (REPA) target. This chapter is the first of two that put this analysis to the test. We begin here with the relatively simpler task of generating small molecules, before moving to much larger molecules in proteins next. In this chapter, we ask: *can REPA accelerate or improve the training of Tabasco [15], a flow-matching transformer-based generator of 3D small molecules?*

Our headline finding is that REPA has little impact on small molecule generation. This is likely because the regime offers REPA little to work with: a near-saturated metric surface, and encoders that carry little structural signal the model does not already have. This null result points to a theme that we develop in subsequent chapters as well. REPA appears to work best where the baseline architecture struggles, but does not appear to offer significant advantage to an already strong learner.

5.1 Experimental setup

5.1.1 Dataset

We train on GEOM-drugs [36], a dataset of drug-like molecular conformers at the sub-900-Dalton scale (§2.4.1). We model heavy atoms only, and augment each molecule with random rotations (eight rotated views per sample at each training step). Each molecule is a small point cloud of typed atoms, often fewer than fifty.

5.1.2 Models

The baseline is the unmodified Tabasco flow-matching transformer (§2.4.1): a 3.7M-parameter non-equivariant model that denoises 3D atom coordinates and atom types jointly. Our REPA variants add a trainable projector that aligns the trunk's hidden states to a frozen molecular encoder, evaluated on the clean (t=1) molecule (§2.2). The trunk carries two hidden-state streams, one for coordinates and one for atom types. We

Property	Value
Base architecture	
Backbone (*trunk*)	Tabasco: 3.7M-param, 16-layer non-equivariant transformer (hidden dim 128, 8 heads, SiLU, cross-attention)
REPA alignment	
Encoder targets	CheMeleon (2D, 2048-d) · MACE (3D, 192-d)
Injection depth ℓ	final (last) layer
Combination mode	additive · tradeoff
Plumbing	fused · separate (CheMeleon; MACE fused only)
Projector	3-layer MLP
Similarity metric sim	cosine similarity
Averaging mode	per atom
Alignment weight λ	0.8
Training & sampling	
Learning rate	2×10^{-3}
Batch size	256
EMA decay	0.999
Sampler	SDE interpolant, 100 integration steps

Table 5.1: Tabasco model and training configuration. The baseline is the trunk alone; the REPA variants add the alignment block. We sweep two encoders and two combination modes against the baseline, with two projector configurations for CheMeleon (MACE was run in the fused configuration only), giving six REPA variants in total. Our analysis centres on the additive combination with fused plumbing.

concatenate them and project the pair through a single head, so the model aligns one *fused* representation rather than two separate ones.

Encoder ablations. The two encoders we study come from the profiling of Chapter 4, and span a useful contrast. *CheMeleon* [30] is a 2D molecular-graph encoder. It is conformer-invariant, so it sees only the bonding graph and not the 3D pose. Meanwhile, *MACE* [29] is a 3D interatomic-potential encoder. It is geometry-aware, but, as the profiling showed, its output is a narrow descriptor of local atomic environments. One carries rich 2D-graph information, the other genuine 3D geometry. We sweep each against the baseline under two combination modes — *additive*, which adds the alignment as a plain regulariser, and *tradeoff*, a convex combination that down-weights the generative term to force the signal in — with two projector configurations for CheMeleon (Table 5.1). Our reading centres on the principled case, additive combination with fused plumbing; we treat tradeoff as a deliberate attempt to wring signal from the loss.

5.1.3 Metrics

We report the standard small-molecule generation metrics of Chapter 3: validity, connectivity, and uniqueness (structural quality); diversity and novelty (set coverage); and the Fréchet ChemNet Distance (FCD) [56], a distributional metric comparing a generated set against a reference. For the diagnosis of §5.3, we additionally fit linear probes to

the frozen trunk's hidden states, recovering molecular descriptors — molecular weight, LogP, ring count, rotatable-bond count — to measure what structural information the representation carries, following the probe principle of Chapter 3.

5.2 Results: no measurable gain

No REPA variant improves on the baseline (Table 5.2). On the saturated quality axes every variant matches the baseline to within about a percentage point. Validity, connectivity, and uniqueness all sit at or near their ceilings, baseline included — as we should expect, since these are largely properties of bond topology, which a strong generator reproduces almost perfectly. The two MACE variants reach perfect validity, with bootstrap intervals that pin it at $[1.0, 1.0]$. There is simply no headroom on these metrics for an intervention to claim.

On FCD — the one metric with room to move — every variant is worse. The baseline records the lowest (best) FCD at 5.61; the REPA variants range from 5.83 to 7.43 (Table 5.2). The ordering is not subtle. The principled CheMeleon variant (additive, fused) is in fact the *worst* of the set, and the MACE variants land in the middle. We caveat this carefully: each variant is a single run, not a multi-seed average, and we do not have per-set FCD intervals to bound the gaps. But the direction is unambiguous — REPA does not improve the only metric that could have improved.

A training-matched view rules out an unfair comparison. The baseline trained roughly twice as long as the REPA variants ($\sim$152k steps over 33 epochs, versus $\sim$69–78k over 14–16). One might worry the variants simply had less time. They did not lack for it on the saturated axes. Validity and connectivity plateau within the first few epochs for every run, baseline and variant alike, and the curves are indistinguishable from then on. The metrics that saturate do so early; the one that does not (FCD) never favours REPA at any point we measured.

The per-step cost is real and one-sided. Adding REPA roughly doubles the cost of a training step for both encoders in their deployed configurations: CheMeleon and cached-MACE run at $\sim$0.74–0.78 s/step, against the baseline's 0.38. Running MACE without its embedding cache is far worse, at $\sim$2.6 s/step (nearly 7$\times$). The intervention is not free, and in this regime it buys nothing.

5.3 Diagnosis

The null has a clean two-part explanation, and both parts trace back to Chapter 4. REPA needs two conditions to help. There must be *headroom*: a gap in what the generator represents or produces, for the alignment to fill. And there must be a *usable teacher*: an encoder whose information both differs from what the trunk already has and transfers to the generative task. In this regime neither condition holds.

Model	Steps	Valid.	Conn.	Uniq.	Div.	Nov.	FCD ↓
Baseline	151k	0.980	0.998	1.000	0.886	0.966	**5.61**
CheMeleon additive (same)	73k	0.967	1.000	0.999	0.891	0.950	5.83
CheMeleon additive (fused)	73k	0.972	0.999	1.000	0.882	0.961	7.43
CheMeleon tradeoff (same)	73k	0.976	0.999	1.000	0.883	0.964	6.49
CheMeleon tradeoff (fused)	78k	0.960	0.999	1.000	0.890	0.942	6.24
MACE additive	69k	1.000	0.995	0.996	0.884	0.977	6.81
MACE tradeoff	69k	1.000	0.999	1.000	0.883	0.982	6.32

Table 5.2: Generation metrics over 1,000 sampled molecules, baseline versus REPA variants. All metrics except FCD are higher-is-better; FCD (Fréchet ChemNet Distance) is lower-is-better, and is the only metric not near its ceiling. The baseline records the best FCD; every REPA variant is worse. *Steps* is the training-step count at evaluation: the baseline trained roughly twice as long as the variants, so the saturated quality metrics are read training-matched (§5.2). One run per variant; bootstrap intervals were computed only for the MACE variants and confirm saturated validity at $[1.0, 1.0]$.

The first condition fails on the metric surface. The quality axes are saturated, as §5.2 showed. A linear probe makes the same point about the representation. Atom-type identity is recovered at 0.999 accuracy from the baseline trunk, and from every variant alike — the model takes atom types as input, so this is trivial. There is no representational gap on this axis for an encoder to close.

The second condition fails on both encoders, for opposite reasons (Table 5.3). The descriptor probe of §5.1.3 compares what each frozen encoder carries against what the trunk already has. CheMeleon decodes the descriptors almost perfectly in isolation (R^2 up to 0.99). But the baseline trunk already decodes them well (0.64–0.92), because they are 2D-graph quantities and the trunk sees the graph. CheMeleon's signal therefore overlaps with what the model already has. Aligning to it adds noise, not information — and it is conformer-invariant, so it cannot guide the 3D geometry that FCD rewards. MACE is the mirror image. It is genuinely 3D, but its output is so narrow that those same descriptors are nearly undecodable from it (R^2 near zero). Its signal differs from the trunk's, but does not transfer.

The lone flicker confirms the reading rather than overturning it. Of all seven models, only MACE in tradeoff mode beats the baseline on the descriptor probe, edging it on three of four targets (including rotatable-bond count, a flexibility-like quantity that depends on geometry). This is the one case where a genuinely 3D teacher, forced in by the convex loss, leaves a trace in the representation. But the trace does not reach generation: that same variant is still worse than the baseline on FCD. A real but tiny representational signal, surfaced only by down-weighting the generative objective, is exactly what a weak-but-3D encoder with no headroom to exploit should produce.

This sharpens the question rather than settling it. The honest conclusion is narrow. Under our setup, REPA does not help small-molecule generation, because the regime offered it neither headroom nor a usable teacher. This is not evidence that REPA

Source	MolWt	LogP	#Rings	#RotBonds
Frozen encoders (the REPA targets in isolation)				
CheMeleon (2D)	0.993	0.988	0.995	0.989
MACE (3D)	0.022	0.307	0.081	−0.014
Dummy	0.822	0.160	0.432	0.345
Generator trunk (student internal representation)				
Baseline	0.920	0.713	0.742	0.636
CheMeleon additive (same)	0.938	0.672	**0.746**	0.600
CheMeleon additive (fused)	0.943	0.669	0.736	0.606
CheMeleon tradeoff (same)	0.930	0.565	0.638	0.532
CheMeleon tradeoff (fused)	0.894	0.633	0.733	0.597
MACE additive	0.935	0.703	0.704	0.582
MACE tradeoff	**0.955**	**0.730**	0.732	**0.666**

Table 5.3: Linear-probe R^2 for four RDKit molecular descriptors, decoded from frozen encoders and from the generator trunk, on held-out molecules. The frozen-encoder block shows the two conditions in miniature: CheMeleon decodes the (2D-graph) descriptors almost perfectly, while MACE — a narrow 3D-geometry encoder — decodes them near zero. The trunk baseline already decodes them well (0.64–0.92), leaving little headroom. Bold marks the best trunk variant per descriptor; only MACE tradeoff beats the baseline, and only by a small margin on three of four targets.

cannot help small molecules in principle — a stronger 3D encoder, or a less saturated metric, could change the picture. But it does suggest that the domain may be the wrong thing to ask about, and the two conditions the right thing. That points to a natural next test: a regime that does offer headroom, with an encoder that does carry transferable structural information. Protein backbones are one such regime, and we turn to them next.

Chapter 6

REPA on protein backbones

In this chapter, we expand the scope of our study on representation alignment from small molecules to much larger ones: proteins. Previously (Chapter 5), our efforts using REPA to generate small molecules yielded little demonstrable improvement, largely because the Tabasco [15] baseline was already saturated on most measures of generation quality. However, de novo backbone design represents a more challenging task, as the data distribution of proteins is significantly larger, and the geometry of these structures is correspondingly more complex. In this context, we ask: *can REPA accelerate or improve the training of Proteina [14], a flow-matching transformer-based protein backbone generator?*

Our headline finding is that REPA accelerates convergence on the two key measures of protein generation quality. This holds across both experimental and synthetic training-data regimes (Figure 6.1). The one exception is designability on synthetic data, which the baseline already saturates under standard sampling (§6.1.1). This chapter characterises *what* happens and attempts to understand *why*. We also pick up the thread from Chapter 3 on how the molecular domain differs from the image domain REPA was built for, and develop a theme throughout: REPA here is a *family* of interventions rather than a single mechanism. Throughout, our aim is to test whether the mechanism transfers at all. We do not search for its best configuration (§6.2.6).

6.1 Experimental setup

We integrate REPA into Proteina [14], a 60M-parameter non-equivariant flow-matching generator of Cα protein backbones (§2.4.2). Following §2.2, we attach a trainable projector at a chosen layer of the generator trunk (the transformer stack) that aligns the model's hidden states to a frozen external encoder.

6.1.1 Datasets

We train in two regimes that span the field's experimental–synthetic spectrum: the experimentally-determined Protein Data Bank [34], and the AlphaFold-predicted AlphaFold Database [35] (Swiss-Prot subset). (We use the manually-curated Swiss-Prot

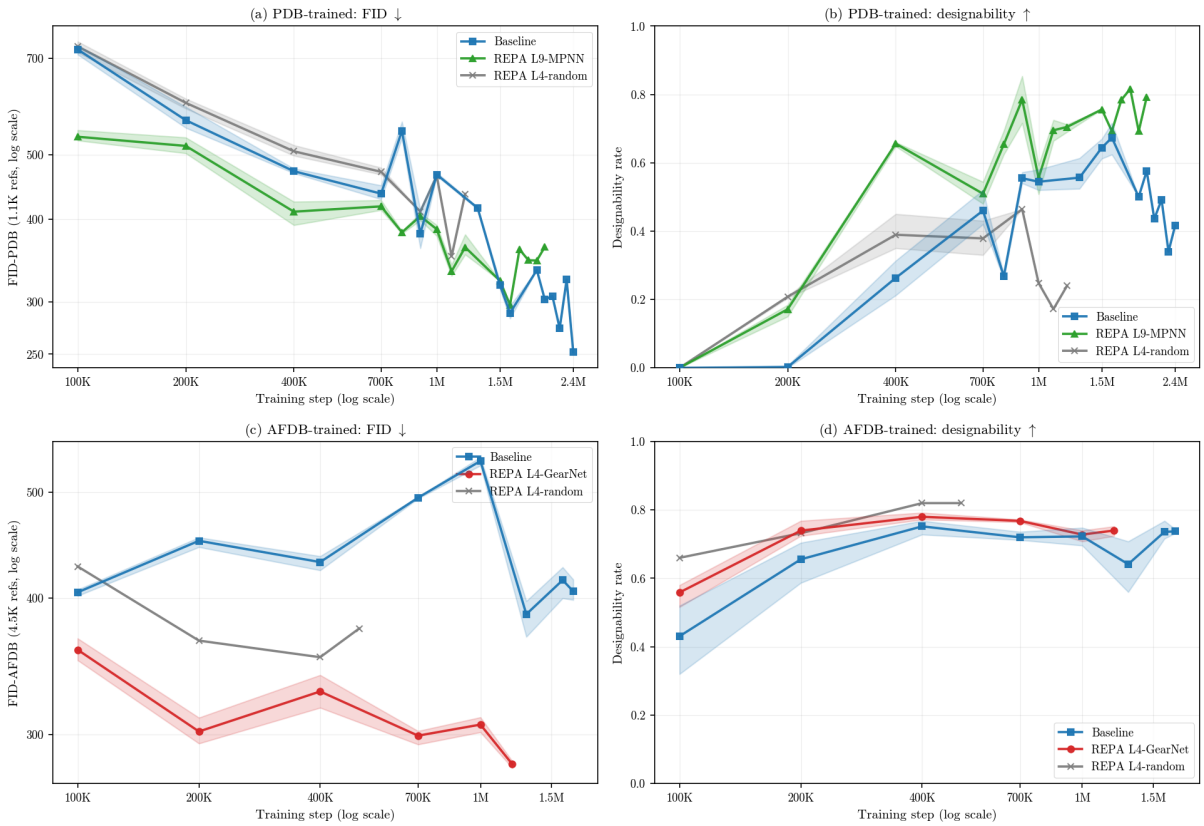

Figure 6.1: **REPA accelerates FID in both data regimes, and designability on experimental (PDB) data.** On synthetic (AFDB) data the designability baseline is already saturated, so we make no acceleration claim there (§6.1.1). The random-encoder control (grey) helps distinguish gains induced by regularisation alone from those induced mechanistically by an encoder's learned structure. While PDB retains a persistent designability gap, its baseline appears to catch up on FID after ≈1M steps; we examine this in §6.2.3. ($n \leq 256$ models, 3 seeds; shaded bands show min/max.)

subset of AFDB as a reproducible stand-in for the Foldseek-clustered TrEMBL subset on which Proteina was trained in the original presentation [14]. This corpus contained data that has since been pruned from AFDB, and is hence inaccessible.)

Following Proteina [14], we train on the subset of chains with at most 256 residues; we represent this length-regime by $n \leq 256$. (We note that this represents only a property of the training datasets, not a bound on the length of the proteins that models trained on these datasets can generate.) Following this filter, the two training sets are comparable in size (PDB: 273,771 / 3,190 / 323 train/val/test chains; AFDB: 229,670 / 4,521 / 235). Appendix A (Table A.1) contains further details on dataset composition. During training, we augment each backbone with a single random global rotation per sample.

Different datasets in this domain carry different biases. Trends in designability offer a clear example, as training on AFDB raises the in-silico-designability floor well above PDB at every setting we tested (§6.2.3). The likely cause is circular. The canonical designability test inverse-folds a generated backbone with ProteinMPNN [24] and re-folds it with ESMFold [23] (Chapter 3). Both of these models share their lineage with the

AlphaFold2 [44] network that produced AFDB in the first place, so AlphaFold-like backbones are plausibly the ones it re-folds most confidently. We do not test this mechanism, but flag the observation so metrics are read with care.

6.1.2 Models

The base Proteina model comprises a stack of ten transformer layers (zero-indexed) that feeds a flow matching objective. Our experiments add a REPA loss to this design, as outlined in Table 6.1.

Encoder and injection-depth ablations. Our narrative concentrates on the two encoders identified as most viable in Chapter 4: *GearNet* [25], a structure encoder pretrained for CATH fold classification (checkpoint released with Proteina [14]); and *ProteinMPNN* [24], which encodes each residue's local inverse-folding environment. Each encoder is attached to the trunk in two layer-depth configurations: layer 4 (middle layer) and layer 9 (last layer). The remaining encoder and depth combinations are collected in Appendix A. Figures presented in this chapter will typically feature the best performing model to serve as an illustrative example.

Separating regularisation from mechanism. A random-weights GearNet attached at layer 4, identical in architecture but never trained, serves as a control architecture in this chapter. We use this model to separate whatever REPA achieves merely due to the regularisation effect of adding an auxiliary loss from what it achieves through the encoder's *learned* structural knowledge. Crucially, a trained encoder at the same layer 4 still outperforms the random one, so the learned-versus-random gap is not an artefact of injection depth (Appendix A).

Hyperparameters. Our defaults largely follow the original image-domain REPA recipe [5].

6.1.3 Metrics

The original hypothesis that motivated REPA for imaging [5] was that alleviating a *representation* bottleneck would improve *generation* quality. We seek to understand if the same mechanism applies in the molecular domain as well. Hence, along with estimating the quality of a model's generated samples, we are also interested in what the model comes to *represent* internally. To this end, we measure three families of metrics, as outlined below: representation quality, representation alignment, and generation quality (Table 6.2).

Representation quality asks what domain-specific information is *linearly decodable* from the trunk's hidden states, following the probe principle of §3. We adapt three probing tasks from the literature, spanning three levels of abstraction:

1. *per-residue inverse-folding (IF) sequence recovery* [27], to understand whether the representation recovers the local chemical environment;
2. *per-residue backbone dihedral angle error* [26], to understand whether it encodes low-level geometry; and

Property	Value
Base architecture	
Backbone (*trunk*)	Proteina (CAFlow): 60M-param, 10-layer non-equivariant Cα transformer (token dim 512, 8 heads)
REPA alignment	
Encoder targets	GearNet (512-d) · ProteinMPNN (128-d) · random-GearNet (control)
Injection depth ℓ	layer 4 (middle) · layer 9 (last)
Projector	3-layer MLP, hidden dim 512
Similarity metric sim	cosine similarity
Averaging mode	per residue
Alignment weight λ	0.5
Training & sampling	
Learning rate	10^{-3}
Batch size	24 ($n \leq 256$)
Weight decay	none
Sampler	SDE, noise scale $\gamma = 0.45$

Table 6.1: Model and training configuration. This chapter presents two encoder targets and two injection depths. Other ablations are deferred to Appendix A.

3. *per-chain CATH fold classification* [26], to understand whether mean-pooled residue representations carry information about global fold structure (the CATH hierarchy is defined in §2.4.2). We headline on architecture (CATH-A), as Class offers arguably too easy a discrimination task given top-1 saturation, and Topology offers too many buckets and too few data points for a meaningful evaluation. However, we report all three levels for completeness, noting that the patterns we observe hold true across them, and motivate our choices in Appendix A (Table A.2).

We note that these three tasks probe largely independent capabilities. Local sequence, local geometry, and global fold are orthogonal to one another, so even a model that is optimised for one may not perform well on the others. Hence, improving performance on all three at once is strong evidence of holistic representation quality.

Representation alignment asks how close one model's representations are to those produced by another — here, the trunk versus a structural encoder. We measure it with CKNNA [61], a mutual-nearest-neighbour kernel alignment between two representations, as used in the original REPA analysis [5] and a recent molecular extension [8]. We use alignment as a diagnostic — a mechanistic lens on *how* REPA might produce its representation gains — rather than a quantity we optimise or build claims on.

Generation quality is what ultimately matters, and we report it in the five groups of Chapter 3: per-sample quality, tertiary structure over the whole set and the designable subset (T-W, T-D), and secondary structure over the same two (S-W, S-D).

A second axis cuts across these groups and recurs through the chapter. *Fidelity* metrics ask how closely the generated distribution matches a reference: FID, the fold Jensen–

Shannon divergence (fJSD), and our secondary-structure divergence (ssJSD-2D). *Diversity* metrics ask how widely the generated set spreads: the fold score (fS), pairwise TM-score, and cluster count. Most of these follow the original presentation in Proteina [14], which introduced distributional (FID-style) metrics to the protein-generation literature.

Building on these, we introduce a new metric, ssJSD-2D, which compares the full (helix-fraction, strand-fraction) secondary-structure distribution between a set of proteins and a reference dataset. A fuller description is given in Appendix A.6. As we will explore in §6.2.4, REPA improves fidelity and whole-set diversity together, but may trade off tertiary-structure diversity *within* the designable subset.

6.1.4 Evaluation protocol

Representation probing. To measure the representation quality of a frozen model checkpoint, we fit linear probes to the hidden states it generates for a batch of clean inputs (t=1). The inverse-folding and dihedral probes are per-residue (a linear classifier and a linear regressor); the CATH probe is a per-chain logistic regression on mean-pooled representations. The classification probes (inverse folding and CATH) are scored as top-1 accuracy. IF is a 20-way problem, while CATH-A is a ∼40-way problem. The dihedral probe reports mean absolute angular error.

Each probe is fit on 1,000 training chains. We evaluate the per-residue probes (inverse folding and dihedral) on a *blinded* set of ∼325 proteins (∼43k residues) held to under 30% sequence identity to all training data, which removes both model- and probe-side leakage. The per-chain CATH probe needs many distinct chains spread across fold classes, which that small slice cannot supply, so we relax it to a *probe-clean* evaluation on a larger validation set of ∼3,190 chains. We detail this scheme, and the model/probe leakage distinction it controls for, in Appendix A.3, and read rank-order and Δ-from-baseline throughout this chapter.

Representation alignment. We measure how close a model's geometry is to an encoder's using the CKNNA [61] metric, using clean (t=1) data from a fixed held-out batch. We compute this metric between the hidden states at every layer in the trunk and three domain-relevant encoders: GearNet, ProteinMPNN, and ESM2 [23].

CKNNA compares two representations using their neighbourhood structure. For each item, it takes the k=10 nearest neighbours in each representation, keeps only the pairs that are neighbours in both, and scores how well the two agree on them. Hence, it measures local structure rather than global geometry. The score is normalised so that self-alignment is 1 and alignment to random features is ≈ 0.

We compute CKNNA per-residue using 10,000 residues. Each value is bracketed by a subsample-without-replacement bootstrap: 50 draws at 80% of the sample. (Sampling with replacement would inflate the metric through duplicate nearest-neighbour pairs.) We note that the absolute CKNNA values we record are small, an order of magnitude

Metric	Definition	Content	Granularity
Representation quality			
IF top-1↑	Inverse-folding sequence recovery (top-1 accuracy)	Semantic content	Per residue
Dihedral MAE↓	Backbone dihedral-angle error	Low-level geometry	Per residue
CATH-A top-1↑	Fold-architecture classification top-1 accuracy (Table A.2)	Fold-level structure	Per chain
Representation alignment			
CKNNA↑ [61]	Mutual-nearest-neighbour kernel alignment	Trunk–encoder similarity	Per residue
Generation quality (Chapter 3)			
Quality			
FID↓	Frchet distance to the reference set (GearNet features)	Distributional fidelity	Whole set
Des↑	Fraction of samples that are designable	Physical realisability	Whole set
Tertiary structure — whole set (T-W)			
fJSD-A↓	Jensen–Shannon divergence vs. reference CATH-A distribution	Fold-distribution fidelity	Whole set
fS-A↑	Spread of generated folds across CATH-architecture classes	Whole-set fold diversity	Whole set
Tertiary structure — designable subset (T-D)			
#Clust↑	Number of TM ≥ 0.5 structural clusters	Designable diversity	Designable subset
pwTM↓	Mean pairwise TM-score, designable subset	Designable diversity	Designable subset
Secondary structure — whole set (S-W)			
ssJSD-2D↓	Jensen–Shannon divergence of the (helix-frac, strand-frac) distribution	Secondary-structure fidelity	Whole set
Secondary structure — designable subset (S-D)			
ssJSD-2D↓	As above, on the designable subset	SS fidelity (designable)	Designable subset
E%des↑	Mean β-strand (E) fraction	β-content	Designable subset

Table 6.2: The three measurement families used in this chapter: representation quality, representation alignment, and generation quality (five reported groups). The whole set is all generated backbones; the designable subset is those passing the designability filter. Arrows give the good direction. Full per-metric results appear in Tables 6.6 and 6.7.

below image-domain REPA [5]. We therefore read the direction and layer structure of the effect, and its separation from the baseline, not its raw magnitude.

Generation sampling. Our generation evaluation follows the Proteina protocol [14], at a reduced sample budget.

For whole-set evaluations, we sample 1,125 backbones per checkpoint — 125 at each length in $\{50, 75, \ldots, 250\}$ — and compute metrics over all of them. Depending on the training regime, we use matching PDB or AFDB reference statistics.

For designability evaluations, 50 backbones at each length $\{50, 100, 150, 200, 250\}$ are inverse-folded with ProteinMPNN (8 sequences each), re-folded with ESMFold, and called designable when the self-consistency RMSD is below 2 Å. The designable-subset metrics (T-D, S-D) are then computed on the backbones that pass.

With this experimental testbed in place, the rest of the chapter investigates what the REPA construction changes with respect to baseline, and how that helps improve generation quality. We now turn to the results.

6.2 Results

6.2.1 REPA improves representation quality asymptotically

At baseline, the flow-matching objective helps the model encode some useful representational information. We probe our models along the three orthogonal axes of §6.1.3 — local chemical environment (inverse folding), local geometry (backbone dihedrals), and global fold (CATH classification, with CATH-A as our headline metric; Figure 6.2). The trained baseline consistently does better than both random features and an untrained Proteina trunk. This suggests that the flow-matching objective does indeed learn to *represent* protein backbones as it learns to *generate* them.

Every trained-encoder variant improves representational capacity, consistently and permanently. Each lifts all three probes substantially above the baseline, from the very beginning of training. That difference then persists. This is an asymptotic gap, not just a head start or an accelerated convergence pattern. This gap confirms one of the hypotheses posited in the original REPA presentation [5]: an alignment target during training can fill the representational headroom left by the flow-matching objective.

Every encoder routes representation gain differently. REPA-GearNet consistently wins the fold probes, while REPA-MPNN wins the per-residue probes (Table 6.3). This pattern follows what we would expect a priori, given the tasks that each encoder has been specifically trained for. This ties into our emerging theme on the multi-factorial nature of representation learning in the molecular domain (Chapter 3). No single encoder captures every factor, so the choice of alignment target is also a choice over the representational factor a model designer wishes to emphasise. We establish this routing on PDB-trained models and report the AFDB results in Appendix A.

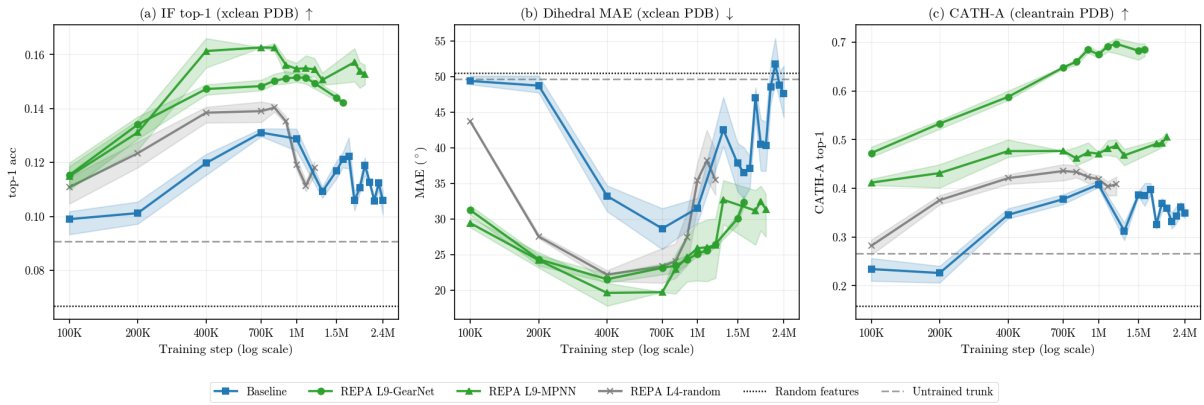

Figure 6.2: **REPA offers a mechanistic effect that lifts the trunk's representation quality asymptotically.** REPA variants open a quality gap over baseline that never closes. The random control gains least, evidence that a trained encoder contributes a genuine mechanistic effect rather than mere regularisation from the auxiliary loss. ($n \leq 256$, PDB-trained; best-layer linear probes; per-residue probes on a blinded cross-database slice, CATH on a probe-clean split; 3 seeds, shaded bands show min/max. See Appendix A.3 for further details and ablations.)

			CATH top-1 acc. ↑		
Variant	**IF top-1 ↑**	**dihedral MAE (°) ↓**	C	A	T
Baseline	0.130	27.6	0.717	0.397	0.303
REPA L4-random	−0.002	+2.7	+0.051	+0.029	+0.017
REPA L9-GN	+0.022	−3.6	+0.163	+0.283	+0.462
REPA L9-MPNN	+0.027	−6.8	+0.085	+0.078	+0.113

Table 6.3: **REPA offers persistent and encoder-routed representation advantages.** Consistent with what each encoder was trained for, REPA-GearNet wins the fold probes (CATH C/A/T), while REPA-MPNN wins the per-residue probes (IF, dihedral). We headline on CATH-A, and report C and T for completeness. ($n \leq 256$ PDB-trained; best-layer linear-probe performance, averaged over the 700K–1.2M training-step window; Δ from baseline; green / darker = improvement/best per probe.)

REPA offers a mechanistic effect beyond just regularisation. Our random-weights control improves on the baseline for the fold probes (CATH), but not for the per-residue ones. Where it helps, this may be because *any* regularisation generically improves representation capacity. It may also be that even an untrained message-passing network imposes some geometric priors that aid the trunk. However, the trained encoders consistently perform far better. This sharpens the case for a genuine mechanistic effect, as regularisation alone is not the whole story.

6.2.2 A diagnostic check on alignment

Following Yu et al. [5] and recent in-domain work [8], we investigate the mechanism that may be driving REPA's representation quality gains by quantifying the representation alignment it induces. Using CKNNA [61], we score how closely our models' per-residue geometry matches that of three domain-specific encoders: GearNet and ProteinMPNN,

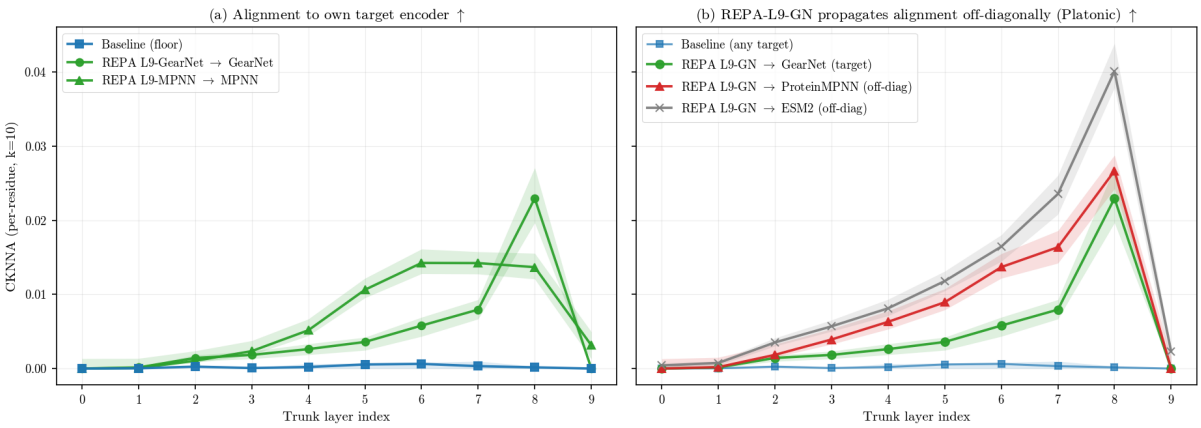

Figure 6.3: **REPA aligns the trunk with domain encoders, including those it was not trained with.** (a) Each REPA variant lifts per-residue alignment to its own target encoder above the baseline floor. (b) REPA induces cross-encoder alignment: for example, a GearNet-aligned trunk also moves toward ProteinMPNN and ESM2. Absolute alignment is small throughout, so we read the pattern, not its magnitude. ($n \leq 256$ PDB-trained, step 1M, t=1.0; per-residue CKNNA, k=10; lines are bootstrap medians, shaded bands are the 5–95% subsample bootstrap interval.)

our two alignment targets in this chapter; and ESM2 [23], which we align to only in our ablations, not in the models presented here. We note that we study alignment only as a diagnostic, and not to make claims about REPA's utility.

REPA lifts alignment above the random floor, albeit at low magnitude. The baseline sits at the random floor at every layer (CKNNA ≈ 0), while REPA separates from it cleanly, suggesting that its hidden states do move towards the encoder's (Figure 6.3a). This serves as a minimal check that the construction changes model behaviour. However, the magnitude of this signal is small. Peak per-residue CKNNA (bootstrap median) is ≈ 0.02–0.05, on a scale where self-alignment is 1. We therefore read the pattern, not its size, and stake no strong claim on these numbers.

Aligning to one encoder nudges the trunk towards others as well. Every REPA variant we tested raises alignment to all three encoders, including those it was not aligned to (full model×encoder matrix in Table A.3, Appendix A.4). A GearNet-aligned trunk, for example, also moves closer to ProteinMPNN and ESM2 [23] (Figure 6.3b), suggesting that REPA-induced alignment is not merely circular self-alignment.

One possible explanation for this phenomenon is the *Platonic representation hypothesis* [61]. This idea suggests models trained on related data converge toward a shared representation. In this framework, REPA may be nudging the trunk toward a common structural representation that several encoders approximate, rather than toward any single encoder.

Variant (last ckpt)	**Acceleration** (@400K)	**Long run** (best)
AFDB FID ↓ — baseline (to 1.7M) @400K 431, best 386 within 2.0M		
GearNet-L4 (1.2M)	+24% (328)	+27% (282 @1.2M)
GearNet-L9 (900K)	+28% (313)	+19% (313 @400K)
MPNN-L4 (1.1M)	−10% (477)	−8% (416 @1.0M)
MPNN-L9 (1.5M)	+18% (352)	+9% (352 @400K)
Random control (500K)	+18% (353)	+9% (353 @400K)
AFDB designability ↑ — baseline (to 1.7M) @400K 0.75, best 0.75 within 2.0M		
GearNet-L4 (1.2M)	+4% (0.78)	+4% (0.78 @400K)
GearNet-L9 (900K)	−4% (0.72)	+3% (0.77 @800K)
MPNN-L4 (1.1M)	−9% (0.68)	−8% (0.69 @700K)
MPNN-L9 (1.5M)	−6% (0.71)	+2% (0.77 @700K)
Random control (500K)	+9% (0.82)	+9% (0.82 @400K)
PDB FID ↓ — baseline (to 2.4M) @400K 472, best 288 within 2.0M		
GearNet-L4 (1.0M)	+7% (440)	−50% (432 @1.0M)
GearNet-L9 (1.6M)	+28% (341)	−11% (319 @700K)
MPNN-L4 (1.6M)	−8% (512)	−14% (329 @1.6M)
MPNN-L9 (2.0M)	+13% (410)	−3% (297 @1.6M)
Random control (1.2M)	−7% (506)	−22% (351 @1.1M)
PDB designability ↑ — baseline (to 2.4M) @400K 0.26, best 0.67 within 2.0M		
GearNet-L4 (1.0M)	+98% (0.52)	+4% (0.70 @700K)
GearNet-L9 (1.6M)	+97% (0.52)	−7% (0.63 @800K)
MPNN-L4 (1.6M)	+124% (0.59)	+5% (0.71 @1.6M)
MPNN-L9 (2.0M)	+149% (0.66)	+21% (0.82 @1.8M)
Random control (1.2M)	+48% (0.39)	−31% (0.46 @900K)

Table 6.4: **REPA accelerates generation quality over the baseline.** In some regimes it also wins the long run. (Values are percentage deltas from the baseline, with the absolute score in parentheses; acceleration is measured at 400K, the anchor of Figure 6.4, and the long run as each variant's best within a common 2.0M-step window; generation scores are means over up to three sampling seeds; $n \leq 256$; green / red = improvement/regression.)

6.2.3 REPA accelerates generation quality by climbing the representation–generation envelope faster

In this subsection, we focus on the holistic *Quality* metrics of our suite, FID and designability (outlined in Table 6.2), to outline our headline results. A deeper analysis of generation quality along tertiary- and secondary-structure decomposition follows in §6.2.4.

REPA gives generation quality an early head start. At 400K steps, every variant already leads the baseline on PDB designability, by +48% to +149%. Most variants also lead on FID in both regimes, with the best at +28%. AFDB designability is the lone exception, for the proxy reason given in §6.1.1 (Table 6.4).

This acceleration is consistent with the representation-bottleneck hypothesis. Yu et al. [5] hypothesise that a model's representation quality and its generation quality are coupled, because it is learning both at once. In our PDB-trained models, this coupling is clear. Across checkpoints, representation and generation quality are positively corre-

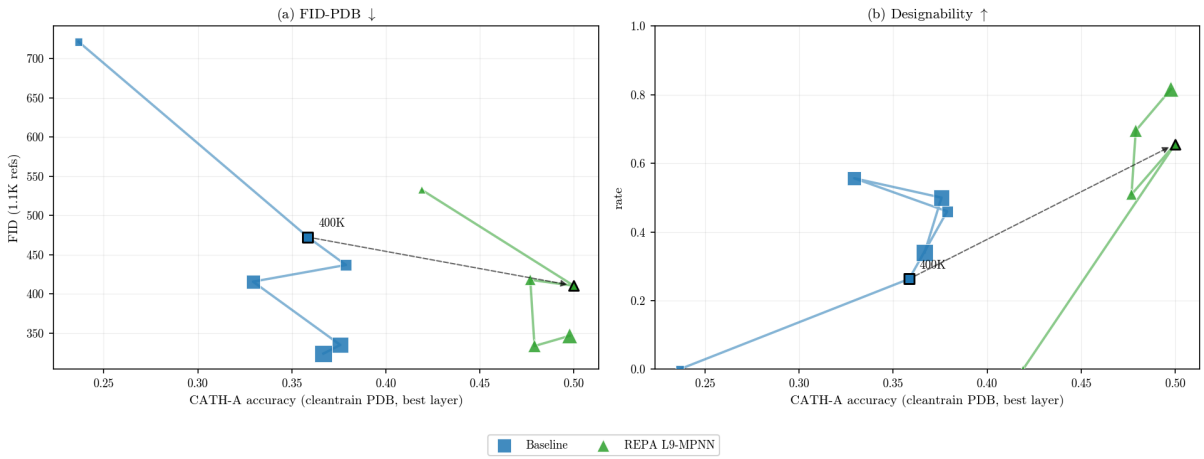

Figure 6.4: **At equal compute, REPA achieves better representation and generation quality.** The dashed arrow marks the matched-compute comparison at step 400K. Against the baseline, the REPA variant has higher fold accuracy (CATH-A) and better generation lower FID (a) and higher designability (b). (Markers are a coarse ≈400K-spaced subset of checkpoints from 100K, with the 400K point bordered; x = student CATH-A accuracy, higher to the right; generation values are means of N=3 sampling seeds; $n \leq 256$, PDB-trained.)

lated, and the correlation survives controlling for training step (Table 6.5). The same coupling holds on AFDB-trained models, and in the same encoder-matched form (Appendix A.5). If the two move together, then securing good representations early should secure good generation early as well.

REPA climbs the representation–generation envelope faster than the baseline. REPA reaches an asymptotically higher representation quality almost as soon as training begins (§6.2.1, Figure 6.2). On the representation–generation curve posited by the bottleneck hypothesis, this places it further along than the baseline at matched compute (Figure 6.4, shown for the strongest PDB variant). Hence, it reaches a given level of generation quality at less training. We interpret REPA as simply traversing this curve faster, rather than finding a new shortcut.

This representation–generation link is encoder-matched, suggesting a causal relationship. Each encoder's generation gain lands on the axis where its representations are strongest. GearNet, the better fold encoder (§6.2.1), gives the larger gain in fold *distribution* (Table 6.6). MPNN, the better local encoder, gives the larger acceleration in *designability* (Table 6.4). A generic regularisation effect would be unlikely to produce this pattern, but we do not claim a formal proof of causality.

Broadly, REPA's lasting advantage tracks the headroom the baseline leaves. Where the baseline plateaus poorly, REPA's lead is large and permanent, as on AFDB FID. Where the baseline is a slow learner, the lead narrows yet still holds, as on PDB designability. Only where the baseline is a genuinely strong learner, as on PDB FID, does it eventually close the gap. AFDB designability, again, leaves little headroom over the baseline. However, as we noted earlier (§6.1.1), this is likely a quirk of the relationship

Representation quality	FID		Designability	
	partial	raw	partial	raw
Inverse folding (top-1)	+0.37	+0.17	+0.79	+0.61
Dihedral MAE	+0.44	-0.03	+0.77	+0.36
CATH-A	+0.22	+0.11	+0.49	+0.38

Table 6.5: **Better student representations track better generation — strongly for designability, moderately for FID.** Partial Pearson correlation controlling for training step, with the raw value alongside; oriented so a positive value means better representation accompanies better generation. (n=59 checkpoints pooled over the baseline, the REPA variants, and the random control; PDB.)

	Quality		T-W		T-D		S-W	S-D	
Variant (700K)	Des↑	FID↓	fJSD-A↓	fS-A↑	#Clust↑	pwTM↓	ssJSD2D↓	ssJSD2D↓	E%des↑
PDB-trained, $n\leq256$, 700K, 3 seeds; FID is FID-PDB (in-distribution).									
Baseline	0.46	437	1.11	5.48	77	0.27	0.35	0.40	0.22
REPA L4-random	−0.08	+34	−0.55	+0.05	−23	−0.10	−0.13	−0.01	−0.11
REPA L4-GN	+0.24	+71	−0.38	+1.63	+1	+0.11	−0.21	−0.12	+0.00
REPA L9-GN	+0.14	−118	−0.57	+2.31	+27	−0.02	−0.19	−0.10	−0.04
REPA L4-MPNN	+0.18	−88	−0.13	+0.48	−4	+0.08	−0.16	−0.14	+0.01
REPA L9-MPNN	+0.05	−19	−0.32	+0.37	+33	−0.01	−0.14	−0.11	−0.07
AFDB-trained, $n\leq256$, 700K, 2–3 seeds (MPNN-L4: 1 seed); FID is FID-AFDB.									
Baseline	0.72	494	2.45	4.59	128	0.22	0.15	0.16	0.15
REPA L4-GN	+0.05	−195	−1.56	+2.27	−35	+0.04	−0.11	−0.07	−0.02
REPA L9-GN	+0.01	−172	−1.29	+2.76	−50	+0.11	−0.12	−0.06	+0.04
REPA L4-MPNN	−0.03	−22	−0.39	+0.03	−1	−0.07	−0.02	+0.05	−0.01
REPA L9-MPNN	+0.05	−139	−0.93	+0.31	+21	−0.03	+0.01	+0.02	−0.04

Table 6.6: **Across the metric suite, almost every REPA variant improves whole-set fidelity and diversity over baseline.** The effect is broad enough that even the random-weights control matches the reference distribution better. However, only the learned encoders also lift per-sample quality (FID and designability). ($n\leq256$, step 700K; baseline absolute, REPA rows show Δ; green / red = improvement/regression; header arrows mark the good direction. Whole-set diversity is read from fS-A; designable diversity from #Clust and pwTM.)

between how designability is measured and how AFDB was generated, and not due to any special merit. Tellingly, even the random control scores highly here.

6.2.4 REPA improves whole-set coverage but may trade off diversity within its designable subset

In this subsection, we investigate how REPA impacts different aspects of generation quality beyond our headline metrics. We read these observations as interesting qualifiers that provide further context to the main convergence result discussed above (§6.2.3).

REPA consistently improves whole-set fidelity and diversity metrics. Under almost every REPA variant tested, the generated set matches the reference fold distribution more closely (fJSD-A) and spreads more widely across fold classes (fS-A); secondary-

Designable pwTM ↓	700K steps			1.0M steps		
Variant	β<10	10–25	$\beta\geq$25	β<10	10–25	$\beta\geq$25
PDB-trained						
Baseline	0.15	0.22	0.50	0.15	0.23	0.13
REPA L9-GN	0.16	0.27	0.73	0.14	0.24	0.87
REPA L9-MPNN	0.16	0.21	0.67	0.26	0.22	0.67
AFDB-trained						
Baseline	0.14	0.29	0.17	0.16	0.26	0.15
REPA L9-GN	0.37	0.31	0.82	0.40*	0.32*	0.76*
REPA L9-MPNN *(falsifier)*	0.15	0.27	0.11	0.15	0.24	0.15

Table 6.7: **A REPA-learned distribution may collapse onto fewer distinct folds within its designable subset.** The effect is strongest in the β-sheet-rich folds. (Designable-subset pairwise TM, lower = more diverse; columns bin the designable subset by β-strand fraction (%); colour marks REPA *vs.* baseline at the same step and bin: red / green = more concentrated/more diverse, $|\Delta|<0.05$ uncoloured. $n\leq256$, single-seed; *900K steps.)

structure fidelity (ssJSD-2D) improves likewise. Curiously, this is true of the random-weights control as well. However, only the learned encoders translate this advantage into higher FID and designability (Table 6.6).

However, a REPA-learned distribution may collapse onto fewer folds within its designable subset. Across most configurations, the designable subset is structurally more concentrated than the baseline's, as measured by the mean pairwise TM-score (pwTM). We use different metrics for whole-set and designable-subset diversity to account for the different set sizes. fS-A is a distributional measure, but the designable subset is far smaller than the whole set, which makes it less interpretable in this context. pwTM is a pairwise average and more robust at smaller sizes. The concentration is especially severe for β-sheet-rich samples (Table 6.7). Crucially, the pattern holds even at matched β-content, i.e. β-strand fraction. Within the most β-rich bin (β-strand fraction $\geq$ 25%), the baseline stays diverse (pwTM$\approx$0.13) while REPA variants collapse onto near-identical folds ($\approx$0.87). Somehow, the REPA construction appears to generate the *same* β-rich folds repeatedly, while the baseline spans many.

On PDB, fold concentration occurs early in training, and the baseline grows out of it. After 700K training steps, the PDB baseline and its REPA variants alike concentrate their β-rich designable samples. However, by 1.0M steps, the baseline has diversified while REPA stays concentrated (Table 6.7). This suggests that REPA does not *engender* the concentration, but rather fails to grow out of the early-training regime that the baseline leaves behind.

Different encoders nudge the model into different distributions. Notably, the AFDB-MPNN variant does not display fold-concentration: its designable subset stays as diverse as the baseline's in every β-bin (Table 6.7). One possible explanation here is that encoder routing leads to different secondary- and tertiary-structure distributions. This

	Sampler setting					
Metric (700K)	ODE	γ=0	γ=0.35	γ=0.45	γ=0.5	γ=1.0
PDB-trained — baseline vs. REPA MPNN-L4						
FID ↓, base	353	782	462	437	425	508
FID ↓, REPA	279	561	371	349	350	424
Des ↑, base	0.04	0.84	0.56	0.46	0.36	0.01
Des ↑, REPA	0.16	0.78	0.66	0.64	0.52	0.11
pwTM (des.) ↓, base	0.26	0.75	0.29	0.27	0.24	0.43
pwTM (des.) ↓, REPA	0.33	0.44	0.36	0.35	0.34	0.34
AFDB-trained — baseline vs. REPA GearNet-L4						
FID ↓, base	220	641	522	494	475	366
FID ↓, REPA	148	586	328	298	292	276
Des ↑, base	0.12	0.91	0.72	0.72	0.69	0.21
Des ↑, REPA	0.35	0.96	0.84	0.77	0.72	0.28
pwTM (des.) ↓, base	0.20	0.69	0.25	0.22	0.22	0.15
pwTM (des.) ↓, REPA	0.19	0.68	0.31	0.26	0.25	0.17

Table 6.8: **REPA tracks the same designability–diversity trade-off as the baseline across sampler settings, at a higher level.** (Absolute values at step 700K; baseline vs. a representative REPA variant per dataset; pwTM is over the designable subset; green / red = REPA better/worse than baseline, small $|\Delta|$ uncoloured; γ=0.45 is Proteina's default; $n \leq 256$.)

MPNN configuration appears to nudge the learned distribution towards helixes, which may be why it escapes this largely sheet-heavy phenomenon (Appendix A.7).

Nevertheless, REPA's whole-set gains persist deep into training. At 1.0M steps, the strongest variant on each dataset keeps its fidelity and coverage lead — REPA-MPNN-L9 on PDB (FID −81, fold-score +0.29) and REPA-GN-L4 on AFDB (−228, +3.68) — even as their distinct designable-fold counts fall below baseline (−51 and −39 clusters).

Does alignment outlive its purpose? Given the early acceleration in generation quality induced by REPA, and the later-stage fold concentration we observe, it is possible that the best approach may be to stop alignment after early training. A similar arc is reported in the image diffusion literature, where REPA is found to help early and is best early-stopped [13]. We do not test such a schedule and make no argument of this form. However, on this reading, an early stop may let REPA diversify like the baseline while keeping the whole-set gains.

6.2.5 REPA's gains hold across sampler noise levels

REPA navigates the same noise–quality trade-off as the baseline. As outlined in §2.1, Proteina samples with an SDE whose noise scale γ trades designability for diversity. REPA does not reshape this trade-off. As γ rises, REPA and the baseline both shed designability and gain diversity together, tracing the same envelope (Table 6.8).

Variant (700K)	ODE designability
PDB-trained	*(baseline = absolute; REPA = Δ vs. baseline)*
Baseline	0.04
REPA L4-random	−0.02
REPA L4-GN	+0.16
REPA L9-GN	+0.04
REPA L4-MPNN	+0.12
AFDB-trained	
Baseline	0.12
REPA L4-GN	+0.23
REPA L9-GN	+0.21
REPA L9-MPNN	+0.10

Table 6.9: **REPA's learned distribution is more designable even without sampling noise.** The ODE limit samples the trunk's learned distribution directly, which suggests REPA learns a better underlying distribution rather than merely benefiting from sampler noise. (ODE designability at step 700K; baseline absolute, REPA rows Δ; green = higher, $|\Delta|<0.025$ uncoloured. Single-seed.)

REPA's gains hold across the sampler's noise range. We report our headline results at γ=0.45, the balance Proteina recommends. The advantage is not specific to that setting. For a representative variant on each dataset, REPA improves designability and fidelity at almost every γ (Table 6.8). The lone exception is the degenerate γ=0 regime, where the baseline saturates designability by repeating a few folds. REPA traces the same trade-off as the baseline, but at a higher level of designability and FID.

REPA raises the deterministic-sampling floor, which points to a better learned distribution. The probability-flow ODE samples the trunk's learned distribution without injected noise (§2.1). As such, it is the closest view of the true learned model distribution, before any correction. Under the baseline architecture, that distribution is barely designable: Des 0.04 on PDB and 0.12 on AFDB. The SDE sampling regime rescues this, with baseline designability climbing to ≈0.84 on PDB at γ=0. However, REPA's learned distribution is already far more designable at ODE, across nearly every learned encoder (Table 6.9). Indeed, only the PDB random control fails to lift it (−0.02). This suggests REPA mechanistically improves the distribution the trunk learns.

6.2.6 What we do not claim

Several tempting claims do not survive our own data, and we flag them here.

- *No asymptotic generation quality win on experimental data.* REPA accelerates PDB convergence, but the lead is transient. The baseline matches REPA's FID plateau by ∼1.4M steps and edges below it by 1.6M (§6.2.3). The durable advantage on fidelity is on synthetic (AFDB) data, where the baseline plateaus poorly.
- *Unclear designability gains on synthetic data.* At the recommended sampler setting, AFDB designability is already biased upwards by construction and near-saturated

for the baseline, so a reliable whole-set gain is hard to establish (§6.1.1). The deterministic ODE floor of §6.2.5 is the exception.

- *No reliable novelty gain.* The signal is too noisy to call in either direction.
- *No general "β-preservation".* The secondary-structure shift is encoder×dataset conditional, and MPNN-AFDB reverses it (§6.2.4).
- *Co-movement, not mediation.* Representation and generation quality move together. We do not claim one formally causes the other, but we note that the pattern is consistent with the representation bottleneck hypothesis (§6.2.3).
- *Unconditional generation only.* Every experiment here is unconditional: we specify a target length but never a fold class or other structural motif. We do not claim these dynamics carry over to the conditional setting (§2.1).
- *No claim of optimality.* We do not search the configuration space for the best setup of layer, weight, or encoder. The question here is whether the mechanism works at all, not how to tune it for maximum gain (§6.1).

6.2.7 Robustness across scale

At smaller scale the learned-versus-random gap narrows, so we anchor headline claims at $n \leq 256$. At $n \leq 128$ the random control helps nearly as much on representation and whole-set metrics. The direction of every gain is consistent across scale, but the margin of learned over random shrinks, so we read $n \leq 128$ as a scale-robustness check rather than a headline regime. Training-dynamics ablations (λ, batch size, projector depth) are reported in Appendix A; length extrapolation — generating proteins longer than the training crop — is the natural next test and remains future work.

6.3 Discussion

This chapter establishes that REPA accelerates Proteina's convergence on generation quality, and shows how the acceleration works. REPA is not a single lever, but a family of encoder-routed interventions. Across all variants we tested, models learn to represent structural characteristics that the flow-matching loss never learns in isolation. It also pulls the trunk's geometry closer to the encoder's, and this alignment generalises across other domain-specific encoders. Learned encoders consistently outperform our random control, a sign that the effect is genuinely mechanistic. Moreover, the representation and generation gains are encoder-matched, strongly suggesting that one runs through the other.

Designability and diversity form a known trade-off, and REPA reaches a better spot on it than the baseline can. Sampler noise slides a fixed model along this trade-off, but REPA enhances the model itself. The REPA architectures we tested improve two things at once about the *whole* distribution learned over training: fidelity to reference

and structural diversity. The clearest sign is the noise-free ODE floor, which REPA lifts, suggesting a genuinely better learned distribution. The gain carries an encoder-specific cost, as some encoders may concentrate the *designable* region of the learned distribution onto fewer folds.

REPA helps most where the baseline struggles most. On experimental data the acceleration is real but transient. The baseline is a strong asymptotic learner and eventually catches up. On synthetic data the baseline plateaus poorly, and there the lead is durable. The intervention is the same in both cases. The asymmetry tracks the baseline's own convergence, not REPA. This is really a data-efficiency statement. It speaks to the motivation of this report directly. REPA's payoff is largest in the abundant-but-synthetic regime the field increasingly leans on. We temper this on designability alone, where the synthetic-data proxy shares lineage with the training distribution (§6.1.1).

These results point past Proteina, to a thread we develop in the conclusion. In a domain whose representation is multi-factorial, choosing an alignment target is a routing decision, not a default. There is no single best encoder. Each one buys a different representational factor — a choice the sampler dial cannot make for you.

We carry this — and the contrast with the small-molecule study of Chapter 5, where the same intervention left no mark — into Chapter 7.

Chapter 7

Conclusions

Bibliography

[1] Jonathan Ho, Ajay Jain, Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. NeurIPS 2020. arXiv:2006.11239.

[2] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, Matt Le. *Flow Matching for Generative Modeling*. ICLR 2023. arXiv:2210.02747.

[3] Peter Holderrieth and Ezra Erives. *An Introduction to Flow Matching and Diffusion Models*. MIT 6.S184 lecture notes, 2025. diffusion.csail.mit.edu.

[4] Yaron Lipman, Marton Havasi, Peter Holderrieth, Neta Shaul, Matt Le, Brian Karrer, Ricky T. Q. Chen, David Lopez-Paz, Heli Ben-Hamu, Itai Gat. *Flow Matching Guide and Code*. 2024. arXiv:2412.06264.

[5] Sihyun Yu, Sangkyung Kwak, Huiwon Jang, Jongheon Jeong, Jonathan Huang, Jinwoo Shin, Saining Xie. *Representation Alignment for Generation: Training Diffusion Transformers Is Easier Than You Think*. ICLR 2025. arXiv:2410.06940.

[6] Jaskirat Singh, Xingjian Leng, Zongze Wu, Liang Zheng, Richard Zhang, Eli Shechtman, Saining Xie. *What matters for Representation Alignment: Global Information or Spatial Structure?* 2025. arXiv:2512.10794.

[7] Chenyu Wang, Cai Zhou, Sharut Gupta, Zongyu Lin, Stefanie Jegelka, Stephen Bates, Tommi Jaakkola. *Learning Diffusion Models with Flexible Representation Guidance*. 2025. arXiv:2507.08980.

[8] Hyosoon Jang, Hyunjin Seo, Yunhui Jang, Seonghyun Park, Sungsoo Ahn. *Boltz is a Strong Baseline for Atom-level Representation Learning*. 2026. arXiv:2602.13249.

[9] Xiangdong Zhang, Jiaqi Liao, Shaofeng Zhang, Fanqing Meng, Xiangpeng Wan, Junchi Yan, Yu Cheng. *VideoREPA: Learning Physics for Video Generation through Relational Alignment with Foundation Models*. 2025. arXiv:2505.23656.

[10] Tri Ton, Jiajing Hong, Chang D. Yoo. *Temporally Aligned Audio for Video with Autoregression*. ICCV 2025. arXiv:2504.05684.

[11] Wei Zhang, Shijie Jin, Hangrui Zhang, Yi Li, Yan Wang. *CrystalREPA: Element-aware Representation Alignment for Crystal Generation*. 2026. arXiv:2605.08960.

[12] Runqian Wang and Kaiming He. *Diffuse and Disperse: Image Generation with Representation Regularization*. 2025. arXiv:2506.09027.

[13] Ziqiao Wang, Wangbo Zhao, Yuhao Zhou, Zekai Li, Zhiyuan Liang, Mingjia Shi, Xuanlei Zhao, Pengfei Zhou, Kaipeng Zhang, Zhangyang Wang, Kai Wang, Yang You. *REPA Works Until It Doesn't: Early-Stopped, Holistic Alignment Supercharges Diffusion Training*. 2025. arXiv:2505.16792.

[14] Tomas Geffner et al. *Proteina: Scaling Flow-based Protein Structure Generative Models*. ICLR 2025. arXiv:2503.00710.

[15] C. Vonessen et al. *TABASCO: A Fast, Simplified Model for Molecular Generation with Improved Physical Quality*. 2025. arXiv:2507.00899.

[16] Marcus Olivecrona, Thomas Blaschke, Ola Engkvist, Hongming Chen. "Molecular de-novo design through deep reinforcement learning". *Journal of Cheminformatics* 9:48 (2017). doi:10.1186/s13321-017-0235-x.

[17] Alex Zhavoronkov et al. "Deep learning enables rapid identification of potent DDR1 kinase inhibitors". *Nature Biotechnology* 37 (2019), pp. 1038–1040. doi:10.1038/s41587-019-0224-x.

[18] Joseph L. Watson et al. "De novo design of protein structure and function with RFdiffusion". *Nature* 620 (2023), pp. 1089–1100. doi:10.1038/s41586-023-06415-8.

[19] John B. Ingraham et al. "Illuminating protein space with a programmable generative model". *Nature* 623 (2023), pp. 1070–1078. doi:10.1038/s41586-023-06728-8.

[20] Amil Merchant et al. "Scaling deep learning for materials discovery". *Nature* 624 (2023), pp. 80–85. doi:10.1038/s41586-023-06735-9.

[21] Claudio Zeni et al. "A generative model for inorganic materials design". *Nature* 639 (2025), pp. 624–632. doi:10.1038/s41586-025-08628-5.

[22] Liang Wang, Chao Song, Zhiyuan Liu, Yu Rong, Qiang Liu, Shu Wu, Liang Wang. *Diffusion Models for Molecules: A Survey of Methods and Tasks*. 2025. arXiv:2502.09511.

[23] Zeming Lin et al. "Evolutionary-scale prediction of atomic-level protein structure with a language model". *Science* 379.6637 (2023), pp. 1123–1130. doi:10.1126/science.ade2574.

[24] J. Dauparas et al. "Robust deep learning–based protein sequence design using ProteinMPNN". *Science* 378.6615 (2022), pp. 49–56. doi:10.1126/science.add2187.

[25] Zuobai Zhang, Minghao Xu, Arian Jamasb, Vijil Chenthamarakshan, Aurelie Lozano, Payel Das, Jian Tang. *Protein Representation Learning by Geometric Structure Pretraining*. ICLR 2023. arXiv:2203.06125.

[26] Arian R. Jamasb, Alex Morehead, Chaitanya K. Joshi, et al. *Evaluating Representation Learning on the Protein Structure Universe*. ICLR 2024.

[27] John Ingraham, Vikas K. Garg, Regina Barzilay, Tommi Jaakkola. "Generative Models for Graph-Based Protein Design". *Advances in Neural Information Processing Systems (NeurIPS)* 32 (2019).

[28] Martin Steinegger, Johannes Söding. "MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets". *Nature Biotechnology* 35.11 (2017), pp. 1026–1028. doi:10.1038/nbt.3988.

[29] Ilyes Batatia et al. "A foundation model for atomistic materials chemistry". *The Journal of Chemical Physics* 163.18 (2024), 184110. doi:10.1063/5.0155322.

[30] Jackson W. Burns et al. "Deep Learning Foundation Models from Classical Molecular Descriptors". arXiv preprint (2025). arXiv:2506.15792.

[31] Michael Heinzinger et al. "Bilingual language model for protein sequence and structure". *NAR Genomics and Bioinformatics* 6.4 (2024), lqae150. doi:10.1093/nargab/lqae150.

[32] Jin Su et al. "SaProt: Protein Language Modeling with Structure-aware Vocabulary". *bioRxiv* (2023). Preprint. doi:10.1101/2023.10.01.560349.

[33] Christoph Schuhmann et al. *LAION-5B: An open large-scale dataset for training next generation image-text models*. NeurIPS 2022 Datasets and Benchmarks. arXiv:2210.08402.

[34] Helen M. Berman et al. "The Protein Data Bank". *Nucleic Acids Research* 28.1 (2000), pp. 235–242. Statistics retrieved Nov 2025, rcsb.org/stats.

[35] Mihaly Varadi et al. "AlphaFold Protein Structure Database in 2024: providing structure coverage for over 214 million protein sequences". *Nucleic Acids Research* 52 (2024), D368–D375. doi:10.1093/nar/gkad1011.

[36] Simon Axelrod and Rafael Gómez-Bombarelli. "GEOM, energy-annotated molecular conformations for property prediction and molecular generation". *Scientific Data* 9, 185 (2022). doi:10.1038/s41597-022-01288-4.

[37] Lorna Richardson et al. "MGnify: the microbiome sequence data analysis resource in 2023". *Nucleic Acids Research* 51 (2023), D753–D759. doi:10.1093/nar/gkac1080. Release statistics retrieved May 2022 (2.4 billion non-redundant sequences): ebi.ac.uk.

[38] Luciano Brocchieri and Samuel Karlin. "Protein length in eukaryotic and prokaryotic proteomes". *Nucleic Acids Research* 33.10 (2005), pp. 3390–3400. doi:10.1093/nar/gki615.

[39] Michael M. Bronstein, Joan Bruna, Taco Cohen, Petar Veličković. *Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges*. 2021. arXiv:2104.13478.

[40] Alexandre Duval, Simon V. Mathis, Chaitanya K. Joshi, Victor Schmidt, Santiago Miret, Fragkiskos D. Malliaros, Taco Cohen, Pietro Liò, Yoshua Bengio, Michael Bronstein. *A Hitchhiker's Guide to Geometric GNNs for 3D Atomic Systems*. 2024. arXiv:2312.07511.

[41] Nathaniel Thomas, Tess Smidt, Steven Kearnes, Lusann Yang, Li Li, Kai Kohlhoff, Patrick Riley. *Tensor Field Networks: Rotation- and Translation-Equivariant Neural Networks for 3D Point Clouds*. 2018. arXiv:1802.08219.

[42] Alexey Dosovitskiy et al. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. ICLR 2021. arXiv:2010.11929.

[43] Jinwoo Kim, Tien Dat Nguyen, Seonwoo Min, Sungjun Cho, Moontae Lee, Honglak Lee, Seunghoon Hong. *Pure Transformers are Powerful Graph Learners*. NeurIPS 2022. arXiv:2207.02505.

[44] John Jumper et al. "Highly accurate protein structure prediction with AlphaFold". *Nature* 596 (2021), pp. 583–589. doi:10.1038/s41586-021-03819-2.

[45] Josh Abramson et al. "Accurate structure prediction of biomolecular interactions with AlphaFold 3". *Nature* 630 (2024), pp. 493–500. doi:10.1038/s41586-024-07487-w.

[46] Johann Brehmer et al. *Does equivariance matter at scale?* NeurIPS 2024. arXiv:2410.23179.

[47] Jason Yim et al. *Fast protein backbone generation with SE(3) flow matching*. 2023. arXiv:2310.05297.

[48] Yeqing Lin and Mohammed AlQuraishi. *Generating Novel, Designable, and Diverse Protein Structures by Equivariantly Diffusing Oriented Residue Clouds*. ICML 2023. arXiv:2301.12485.

[49] Tuan Le et al. *Navigating the design space of equivariant diffusion-based generative models for de novo 3D molecule generation*. ICLR 2024. arXiv:2309.17296.

[50] Clement Vignac et al. *MiDi: Mixed Graph and 3D Denoising Diffusion for Molecule Generation*. ECML PKDD 2023. arXiv:2302.09048.

[51] Ross Irwin et al. *Efficient 3D Molecular Generation with Flow Matching and Scale Optimal Transport*. 2024. arXiv:2406.07266.

[52] National Institutes of Health. *Regulatory Knowledge Guide for Small Molecules*. NIH SEED, 2024. seed.nih.gov.

[53] Encyclopaedia Britannica. *What is the difference between a peptide and a protein?* britannica.com.

[54] Gisbert Schneider. "Automating drug discovery". *Nature Reviews Drug Discovery* 17 (2018), pp. 97–113. doi:10.1038/nrd.2017.232.

[55] Martin Buttenschoen, Garrett M. Morris, and Charlotte M. Deane. "PoseBusters: AI-based docking methods fail to generate physically valid poses or generalise to novel sequences". *Chemical Science* 15 (2024), pp. 3130–3139. arXiv:2308.05777.

[56] Kristina Preuer, Philipp Renz, Thomas Unterthiner, Sepp Hochreiter, and Günter Klambauer. "Fréchet ChemNet Distance: A Metric for Generative Models for Molecules in Drug Discovery". *Journal of Chemical Information and Modeling* 58.9 (2018), pp. 1736–1741. doi:10.1021/acs.jcim.8b00234.

[57] G. Richard Bickerton et al. "Quantifying the chemical beauty of drugs". *Nature Chemistry* 4 (2012), pp. 90–98. doi:10.1038/nchem.1243.

[58] Pascal Notin et al. “Machine learning for functional protein design”. *Nature Biotechnology* 42 (2024), pp. 216–228. doi:10.1038/s41587-024-02127-0.

[59] Yeqing Lin, Minji Lee, Zhao Zhang, and Mohammed AlQuraishi. “Out of Many, One: Designing and Scaffolding Proteins at the Scale of the Structural Universe with Genie 2”. *arXiv preprint* arXiv:2405.15489 (2024). arXiv:2405.15489.

[60] Christine A. Orengo, A. D. Michie, S. Jones, David T. Jones, M. B. Swindells, Janet M. Thornton. “CATH — a hierarchic classification of protein domain structures”. *Structure* 5.8 (1997), pp. 1093–1108. doi:10.1016/S0969-2126(97)00260-8.

[61] Minyoung Huh, Brian Cheung, Tongzhou Wang, Phillip Isola. *The Platonic Representation Hypothesis*. ICML 2024. arXiv:2405.07987.

Appendix A

Technical details

A.1 Dataset composition and split overlap

Table A.1 summarises the two datasets used in Chapter 6. Both training sets are α/β-dominated and span a substantial fraction of CATH architectures (44.2% PDB, 61.8% AFDB), which is what makes the per-chain CATH fold probe meaningful. Neither random split is sequence-clustered: 98.3% of PDB and 92.2% of AFDB validation chains have a $\geq$30%-identity homolog in their own training set, so absolute in-house scores are inflated and we read rank-order and Δ-from-baseline instead. The cross-database overlap is far lower — only 62.1% of AFDB validation chains have a $\geq$30%-identity hit in PDB training — so a homology-filtered AFDB set serves as a cleaner probe set for PDB-trained models (Chapter 3).

A.2 Choice of CATH fold-probe level

We report CATH *architecture* (CATH-A) as the headline fold-probe level throughout Chapter 6. Table A.2 justifies that choice: Class is too coarse (four buckets, one already near-saturated), Topology too finely-bucketed to probe reliably, and Architecture sits between — discriminative but learnable.

Property	**PDB**	**AFDB (Swiss-Prot)**
Origin	Experimental (X-ray, cryo-EM)	Predicted (AlphaFold2, v6)
Residue-length crop n	$\leq$ 256 residues	
Train / val / test chains	273,771 / 3,190 / 323	229,670 / 4,521 / 235
CATH coverage (train)	44.2%	61.8%
Val$\leftrightarrow$train, $\geq$30% id.	98.3%	92.2%
AFDB-val$\leftrightarrow$PDB-train, $\geq$30% id.	62.1%	

Table A.1: Properties of the two datasets we study, at the headline $n\leq$256 regime. Sequence identity is computed with MMseqs2 [28] (`easy-search`, $\geq$30% identity over $\geq$80% alignment coverage).

CATH level	#classes	baseline probe acc.	verdict
C (class)	4	0.733	one class (α/β) is 55% of the data; coarse, already near-saturated — *gameable*
A (architecture)	∼40	0.407	balanced and discriminative, with headroom to move — **our headline fold metric**
T (topology)	∼1400	0.301	long-tailed (top fold ∼18%; only ∼89 classes populated enough to probe); per-class accuracy is unreliable — *too many buckets*

Table A.2: **We read fold-level representation quality off CATH *architecture* — Class is too coarse to discriminate, Topology too finely-bucketed to probe reliably.** Class has 4 buckets (one at 55%, baseline already 0.73); Topology has ∼1400 imbalanced buckets; Architecture sits between — enough classes to be meaningful, enough examples per class to be learnable — so CATH-A is our headline fold metric (C and T are reported in the appendix). ("Baseline probe acc." is a linear probe on the un-aligned trunk.)

A.3 Leakage-controlled probe evaluation

The random train/val split (§A.1) is not sequence-clustered, so probe scores are exposed to two distinct leakages. *Model leakage* is asymmetric: a model trained on the leaky split has effectively seen near-duplicates of the evaluation proteins, an advantage a differently-curated model (such as an externally-pretrained checkpoint) does not share. *Probe leakage* is symmetric: a probe fit on training chains near-identical to its evaluation set can shortcut by matching sequences rather than learning generalisable structure, inflating every model's absolute score.

We control these by filtering the evaluation. A *probe-clean* split removes probe leakage alone — the probe's training chains are filtered to have no ≥30% sequence-identity hit to the evaluation set, which is otherwise the full validation set. A *blinded* split removes both — the evaluation proteins are held to under 30% identity to all training data (here, a cross-database set of AFDB chains with no such hit in either the PDB or AFDB training pool, ∼325 proteins). We use the blinded split for the per-residue probes (inverse folding, dihedral), where the ∼325 proteins still yield ∼43k residues — full statistical power. The per-chain CATH probe is different: fold classification needs many distinct chains spread across populated fold classes, and the blinded slice is too small and too class-sparse for this, so we relax it to the probe-clean split on the fuller validation set (∼3,190 chains). Comparing the splits also decomposes the leakage — the dirty-to-probe-clean drop isolates probe leakage and the probe-clean-to-blinded drop isolates model leakage — and the REPA-over-baseline gap survives in every regime.

	GearNet	ProteinMPNN	ESM2
Baseline	0.6^{L6}	0.9^{L7}	2.7^{L8}
REPA GN-L4	$\mathbf{2.6^{L4}}$†	2.9^{L6}	8.2^{L8}
REPA GN-L9	$\mathbf{22.9^{L8}}$†	26.7^{L8}	40.1^{L8}
REPA MPNN-L4	4.8^{L7}	$\mathbf{16.6^{L7}}$†	19.8^{L5}
REPA MPNN-L9	5.0^{L7}	$\mathbf{14.2^{L6}}$†	14.2^{L7}

Table A.3: **Every REPA variant raises per-residue alignment to all three encoders, including the two it was never aligned to.** Across-layer peak per-residue CKNNA (bootstrap median, $\times 10^3$), with the peak trunk layer as a superscript; k=10, step 1.0M, $n \leq 256$ PDB-trained. The † marks each model's own alignment target. Every REPA row exceeds the baseline in every column, the Platonic-convergence signature discussed in §6.2.2. Absolute values are small (an order of magnitude below image-domain REPA), so we read the pattern, not the magnitude.

A.4 Representation-alignment matrix

The alignment diagnostic in §6.2.2 reports peak per-residue CKNNA for a single illustrative variant. Table A.3 gives the full model×encoder matrix behind that reading. It confirms the off-diagonal generalisation we describe there. Aligning the trunk to one encoder raises its CKNNA to the other two as well, not just the alignment target. The peak sits at the last trunk layers (L7–L8 here). The absolute values stay an order of magnitude below image-domain REPA, so we read the direction and layer pattern rather than the raw magnitude.

A.5 Representation–generation coupling on AFDB

Section 6.2.3 establishes the representation–generation coupling on PDB-trained models (Table 6.5). Table A.4 repeats the analysis for AFDB-trained models. The coupling holds here too, and in the same encoder-matched form. Local probes (inverse folding, dihedral) track designability, while the fold probe (CATH-A) tracks FID. The off-axis pairs are weak or absent, the signature of an encoder-routed effect rather than a generic one. We read AFDB designability with the proxy caveat of §6.1.1, so we lean on the FID column and the routing pattern rather than the absolute designability correlation.

A.6 The ssJSD-2D secondary-structure metric

Proteina's secondary-structure metric compares the *mean* helix and strand content of a generated set against a reference. This collapses each set to a single point, so it cannot see variance, bimodality, or tails — a model that matches the average composition while collapsing its spread scores the same as one that reproduces it. ssJSD-2D removes this blind spot by comparing the full joint distribution.

For each backbone we annotate every residue as helix, strand, or coil with a Cα-only

Representation quality	FID		Designability	
	partial	raw	partial	raw
Inverse folding (top-1)	-0.17	-0.16	+0.50	+0.50
Dihedral MAE	+0.03	-0.03	+0.37	+0.09
CATH-A	+0.42	+0.43	+0.29	+0.34

Table A.4: **On AFDB too, better student representations track better generation.** The AFDB counterpart to Table 6.5: partial Pearson controlling for training step (raw alongside), oriented so positive means better representation accompanies better generation. Generation is FID-AFDB and designability; representation probes are read at n_{train}=1000 on the cross-database blinded set. (n=40 AFDB checkpoints pooled over the baseline, the REPA variants, and the random control.)

secondary-structure assignment (P-SEA, as implemented in Biotite) and record the fraction of residues in helix (f_H) and in strand (f_E). A set of N backbones is then a cloud of N points in the (f_H, f_E) plane. We bin that plane into a 5×5 grid over $[0, 1]^2$, form normalised two-dimensional histograms for the generated and reference sets, and report the Jensen–Shannon divergence between them (lower is better). Binning the joint distribution, rather than comparing marginal means, lets the metric reward a model for reproducing the *shape* of the secondary-structure distribution and penalise mode collapse onto, say, all-α backbones.

A.7 Secondary-structure composition of the designable subset

The encoder-routing reading in §6.2.4 — GearNet shifting the designable subset toward β-sheet, MPNN on AFDB away from it — rests on the secondary-structure composition of the designable backbones (Table A.5). The shifts are small in absolute terms but seed-consistent: on AFDB the GearNet-L9 and MPNN-L9 strand and helix ranges do not overlap across seeds, in opposite directions. The main-text E%des figures (Table 6.6) are the per-seed means of the strand column; we report the spread here rather than in the main table to avoid clutter.

Variant (700K, γ=0.45)	Strand f_E	Helix f_H	Seeds
PDB-trained			
Baseline	0.22 ± 0.02	0.48 ± 0.03	3
REPA L4-random	0.11 ± 0.01	0.61 ± 0.02	3
REPA L4-GN	0.22 ± 0.01	0.40 ± 0.02	3
REPA L9-GN	0.18 ± 0.02	0.48 ± 0.03	3
REPA L4-MPNN	0.23 ± 0.01	0.39 ± 0.03	3
REPA L9-MPNN	0.15 ± 0.01	0.54 ± 0.02	3
AFDB-trained			
Baseline	0.16 ± 0.01	0.47 ± 0.01	2
REPA L4-GN	0.14 ± 0.00	0.47 ± 0.01	2
REPA L9-GN	0.19 ± 0.01	0.38 ± 0.03	3
REPA L9-MPNN	0.11 ± 0.00	0.55 ± 0.01	2

Table A.5: **Secondary-structure composition of the designable subset (step 700K, γ=0.45).** Mean strand (f_E) and helix (f_H) fractions over the designable backbones, $\pm$ half the seed range. GearNet shifts the designable subset toward strand and away from helix; MPNN on AFDB does the reverse. The AFDB GearNet-L9 vs. MPNN-L9 ranges are non-overlapping on both axes. The strand column is the absolute form of the E%des column in Table 6.6, whose entries are the per-seed means reported here. (AFDB MPNN-L4 omitted: no multi-seed γ=0.45 data.)